

SAPIENZA
UNIVERSITÀ DI ROMA

Seismic Viewer: Design, Implementation and Evaluation of an Interactive 4D Spatio-Temporal Visualisation System for Seismological Data

Faculty of Information Engineering, Computer Science and Statistics
Applied Computer Science and Artificial Intelligence

Alessio Curcio
ID number 2120317

Supervisor
Danilo Avola

Co-supervisor
Daniele Bailo

Academic Year 2025/2026

Seismic Viewer: Design, Implementation and Evaluation of an Interactive 4D Spatio-Temporal Visualisation System for Seismological Data
Bachelor's thesis. Sapienza University of Rome

© 2026 Alessio Curcio. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: alessio.curcio04@gmail.com

To my family, colleagues, and friends, by my side today or carved into my memories,
whose support is etched in the surface of every page,
shaping every step of this path.

Acknowledgments

First and foremost, I would like to express my gratitude to my academic supervisor, Prof. Danilo Avola, for his availability and for accepting to supervise my internship project. I am equally indebted to my internship supervisor, Dr. Daniele Bailo, for the trust he placed in me by welcoming me under his wing and for providing continuous guidance and mentorship during this project.

I am grateful to Giada and Rossella for their collaboration. Their constant readiness to share their knowledge and their moral support were a valuable help throughout this work.

A special thanks goes to Alessandro and all my friends from the "bunker". Thank you for being by my side through the many highs and lows of these past three years of my bachelor's degree. I also want to extend my sincere appreciation to Gianmaria for his patience and help during our countless hours of study together.

Finally, my heartfelt gratitude goes to my family for their patience, emotional support, and for always believing in me.

Abstract

This thesis presents the design, implementation, and evaluation of **Seismic Viewer**, an interactive desktop application for the spatial and temporal visualization of seismic data. The system was developed during an internship at the INGV to address a limitation of existing tools: the lack of software capable of displaying large earthquake datasets while integrating spatial clustering and geological plane estimation.

Traditional seismic analysis often relies on two-dimensional maps or static plots that do not show the evolution of earthquake sequences over time. Seismic Viewer addresses this limitation with an architecture that combines a Python backend for data processing with a WebGL frontend for 3D graphics in an interactive desktop window.

The application provides four display modes: 2D maps, 3D static views, 4D animated sequences, and cluster analysis with fault plane estimation. It imports data from local CSV files, the INGV GOSSIP API, ground uplift measurements, and seismic tomography. Clustering algorithms, automatic parameter tuning, plane fitting, and a label matching method for color stability over time help identify fault structures.

This thesis describes the complete development process from initial requirements to system architecture, testing, and optimization. The application was tested with earthquake data from the Campi Flegrei area, showing smooth interaction with large seismic catalogs. The binary streaming protocol reduced data payload by 80% compared to JSON, and the plane fitting pipeline estimated strike and dip angles with a mean error below 0.6° under realistic noise conditions.

Contents

1 Introduction **1**
1.1 Motivation and Context . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives and Scope . . . 2
1.4 Internship Context . . . 2
1.5 Document Structure . . . 3

2 Background and Related Work **4**
2.1 Seismological Fundamentals . . . 4
2.2 The Campi Flegrei Volcanic Complex . . . 5
2.3 Existing Seismic Visualisation Tools . . . 5
2.4 Machine Learning in Seismology . . . 7
2.5 Technology Choices and Justifications . . . 8
2.5.1 Architectural Trade-Off: pywebview vs. Electron . . . 8
2.5.2 Rendering Engine Selection: Plotly.js vs. Low-Level Graphics APIs . . . 9
2.5.3 Backend Server Framework: FastAPI vs. Traditional Frameworks . . . 9

3 System Architecture **10**
3.1 Architectural Overview . . . 10
3.2 Backend Architecture . . . 11
3.2.1 Application Entry Point & Bridge Engine . . . 11
3.2.2 Session Store & FastAPI Middleware . . . 12
3.3 Frontend Architecture . . . 12
3.3.1 Single-Page Application & UI/UX Design . . . 12
3.4 Data Flow Pipeline and Protocols . . . 14
3.4.1 Binary Data Transfer Protocol . . . 15
3.5 Threading and Concurrency Model . . . 15

4 Data Processing and Geospatial Engine **16**
4.1 Pipeline Architecture and Multi-Source Ingestion . . . 16
4.2 CSV Ingestion and Schema Detection . . . 17
4.3 Data Sanitisation and Validation . . . 18
4.4 Online Data Import: The INGV GOSSIP API . . . 18
4.5 Geospatial Computations and Mesh Generation . . . 19

4.5.1 UTM Zone 33N to WGS84 Conversion . . . 20
4.5.2 Optimised Map Mesh and Caching Architecture . . . 20
4.6 Pre-computation and Memory Layout Transformation . . . 20

5 Visualisation Modes 21
5.1 2D Geospatial Map View . . . 21
5.2 3D Static View . . . 23
5.3 3D Animated View (4D Visualisation) . . . 26
5.4 3D Cluster Analysis View . . . 27
5.4.1 Cluster Visualisation and Interactive Tuning . . . 27
5.4.2 Sub-Clustering . . . 28
5.5 Additional Data Layers & View Caching . . . 31
5.5.1 Auxiliary Layers: Tomography, Uplift, and Stations . . . 31
5.5.2 View Caching and Navigation . . . 32

6 Clustering and Structural Analysis Algorithms 33
6.1 Coordinate Projection and Spatial Clustering Methods . . . 33
6.1.1 DBSCAN, MiniBatchKMeans, and Cylinder Partitioning . . . 33
6.2 Automatic Parameter Tuning . . . 34
6.3 Structural Plane Fitting and Geometric Synthesis . . . 35
6.3.1 RANSAC Robust Plane Estimation . . . 35
6.3.2 SVD Refinement and Strike/Dip Mechanics . . . 35
6.4 Temporal Stability, Parallel Execution, and Complexity . . . 36
6.4.1 Temporal Stability via Global Label Matching . . . 36
6.4.2 Parallel Execution Architecture . . . 37
6.4.3 Asymptotic Complexity Analysis . . . 37

7 Development Process and Challenges 38
7.1 Development Timeline & Iterative Methodology . . . 38
7.2 Key Technical Challenges and Solutions . . . 38
7.2.1 Rendering Performance and UX Optimisation . . . 39
7.2.2 Backend Architecture, Concurrency, and Algorithmic Resilience 40
7.3 Code Quality and Architecture . . . 41
7.4 Validation and Quantitative Performance Assessment . . . 41
7.4.1 Experimental Benchmarks: Serialization and Streaming . . . 42
7.4.2 Clustering Latency and Algorithmic Throughput . . . 42
7.4.3 Synthetic Validation of Structural Plane Extraction . . . 42

8 Conclusions and Future Work 44
8.1 Summary of Contributions . . . 44
8.2 Future Research and Architectural Extensions . . . 44
8.3 Concluding Remarks . . . 45

A Supplementary Technical Details 46
A.1 Seismic Data Structures . . . 46
A.1.1 Input Catalog Format . . . 46
A.1.2 Internal Application State . . . 46

A.2 Network Protocol and Message Schemas 48
A.2.1 WebSocket Message Specification 48
A.2.2 Core Default Configuration Parameters 49

List of Figures

3.1 Client-server architecture of Seismic Viewer with dedicated communication channels. 11
3.2 The Seismic Viewer single-page application structure, showing initial data loading (a) and the multi-mode analytical selection hub (b). . . 13
3.3 Linear data pipeline from input parsing to WebGL drawing. 14
3.4 Sequence diagram of asynchronous execution and non-blocking streaming pipeline. 14
3.5 Concurrency hierarchy linking UI, server thread, and analysis worker threads. 15

4.1 Data ingestion pipeline of Seismic Viewer, showing data sources (Local CSV, INGV GOSSIP API, and Tomography or GNSS data), the validation pipeline, and data conversion into memory arrays and binary buffers. 17
4.2 The INGV GOSSIP API import dialog for configuring network criteria, temporal bounds, and quality thresholds. 19

5.1 The 2D Geospatial Map View in the Seismic Viewer. This workspace allows users to filter earthquake data (a) and combine it with advanced geophysical overlays, such as tomographic velocity maps (b) and ground uplift measurements (c). 23
5.2 The 3D Static View workspace. The user can explore the underground earthquake distribution (a) and combine it with 3D tomography models (b) and surface deformation data (c). 25
5.3 The 4D Animated Spatio-Temporal View workflow. Users configure parameters via the dialog (a), explore time-evolving seismicity (b), and observe automated fault plane tracking (c). 27
5.4 Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 1). 29
5.4 Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 2). 30

7.1 Development milestones highlighting the iterative implementation methodology. 38

List of Tables

2.1 Comparison of existing seismic visualisation tools against Seismic Viewer. 7
2.2 Summary of the selected technology stack and rejected alternatives. 8

3.1 REST API endpoints and WebSocket message protocol. 11
3.2 Binary buffer layout (Float32). 15

6.1 Comparison of spatial clustering approaches in Seismic Viewer. . . . 34
6.2 Asymptotic time and space complexity of the analytical methods. . . 37

7.1 Serialization benchmark: JSON vs. Float32 Binary buffer. 42
7.2 Clustering execution latency benchmark across dataset scales. 42
7.3 Synthetic plane estimation accuracy under Gaussian positional noise. 43

A.1 Expected schema for input seismic catalogs. 47
A.2 Internal data representation in the processing pipeline 47
A.3 Default parameters and operational thresholds across the analytical engine 49

Chapter 1

Introduction

1.1 Motivation and Context

Seismology is a data-intensive field within Earth sciences. Seismic networks continuously record events, producing datasets that describe subsurface processes such as tectonic stress and fluid movement. Interpreting earthquake data in real time is important for volcanic and seismic risk assessment.

Traditionally, earthquake catalogs have been analyzed through two-dimensional maps, cross-sections, and time plots. These representations do not fully capture the spatial and temporal distribution of seismic events. An earthquake sequence moving along a fault at depth may look like a diffuse group in map view, but show a clear planar structure when examined in three dimensions and animated over time. This limitation of conventional tools motivated the development of **Seismic Viewer**.

The project was developed during an internship at the Istituto Nazionale di Geofisica e Vulcanologia (INGV). Researchers studying the Campi Flegrei area needed a tool to display spatial distributions, temporal evolution, and structural connections in an interactive desktop application.

1.2 Problem Statement

Despite the availability of numerous Geographic Information System (GIS) platforms and seismological software packages, a significant technological gap persists. Existing tools generally fall into one of two categories:

(a) **General-purpose GIS platforms** (e.g., QGIS, ArcGIS): These offer geospatial analysis capabilities but are not designed for rendering dynamic 3D point clouds with seismic metadata in real time. Their architectures prioritize cartographic precision over interactivity, leading to delays when handling datasets with tens of thousands of hypocentres. From a software architecture perspective, traditional GIS tools often use synchronous data processing and text formats like JSON. When handling large datasets with tens of thousands of points, this causes the interface to freeze due to high memory use and parsing delays.

(b) **Specialised seismological software** (e.g., Seisan, GMT, ObsPy): These provide robust analytical functionality but typically produce static outputs, such as static figures for print rather than interactive explorations. The learning curve is steep, requiring command-line proficiency, and the visualisation paradigm remains fundamentally two-dimensional.

Neither category adequately addresses the need for a tool that unifies *real-time 3D rendering*, *temporal animation*, *machine learning-based clustering*, and *structural geological analysis* within a single, accessible desktop application. Seismic Viewer was designed to fill this gap.

1.3 Objectives and Scope

The primary objectives of this project are:

1. **Interactive 4D Visualisation**: Develop a system capable of rendering seismic hypocentres in three spatial dimensions plus time, with smooth animations and real-time filtering.
2. **Automated Structural Analysis**: Integrate spatial clustering algorithms (DBSCAN, K-Means) and plane fitting (RANSAC + SVD) to automatically identify seismogenic structures.
3. **Multi-Source Data Integration**: Support ingestion from local CSV files, the INGV GOSSIP online API, seismic tomography datasets (V_p/V_s velocity ratios), and Global Navigation Satellite System (GNSS) ground deformation data.
4. **Performance at Scale**: Ensure sub-second interactive response times for large datasets, through a custom Float32 binary transfer protocol, an asynchronous communication design (bridge, HTTP REST, and WebSockets), GPU-accelerated rendering, and efficient caching strategies.
5. **Accessible User Experience**: Provide a modern, intuitive interface that does not require programming expertise, enabling geophysicists to focus on interpretation rather than tool operation.

This report covers the development process of the application, from the initial design to testing and evaluation. The system was evaluated using earthquake data from the Campi Flegrei caldera. In addition to practical usability, this thesis explains the software solutions used to connect Python calculations with interactive 3D web graphics. The following chapters present the domain background, the system architecture, the data processing pipeline, the visualisation modes, the algorithmic foundations, the development process, and the conclusions drawn from this work.

1.4 Internship Context

This work was conducted as part of a university internship. The project was developed collaboratively with Giada Alfonsi, a PhD student in geology at Roma

Tre University, under the supervision of Daniele Bailo, a software engineer at the INGV.

An iterative development approach was adopted, progressively expanding the software's capabilities over the duration of the internship. Rather than following a rigid specification, each development cycle was shaped by the continuous feedback and domain expertise provided by Giada Alfonsi, Professor Guido Giordano (volcanologist and professor at Roma Tre University), researchers at the INGV Osservatorio Vesuviano, and Daniele Bailo. The primary phases of the project were as follows:

- **Phase 1**: Initial prototype, basic 3D visualisation, and local CSV ingestion.
- **Phase 2**: Implementation of K-Means clustering and connection with the web view bridge.
- **Phase 3**: Redesign of the user interface and integration of the INGV GOSSIP API for data import.
- **Phase 4**: Optimization using binary data transfer, cylinder partitioning, sub-clustering, and automatic tuning of DBSCAN parameters with K-distance analysis.
- **Phase 5**: Integration of seismic tomography, ground uplift layers, and monitoring station positions.
- **Phase 6**: Modular refactoring of the package, setup of FastAPI and WebSockets, and automated tests.

This collaborative and iterative feedback process proved essential for aligning the software with real analytical workflows, as several key features (such as the cylinder grid partitioning and the sub-clustering interface) emerged directly from practical requirements and discussions during development.

1.5 Document Structure

The remainder of this thesis is organised as follows. Chapter 2 introduces the seismological background and existing tools. Chapter 3 describes the system architecture and the communication between frontend and backend. Chapter 4 explains data loading, processing, and optimizations. Chapter 5 presents the visualisation modes and additional data layers. Chapter 6 details the clustering and fault plane fitting algorithms. Chapter 7 discusses the development process and the main challenges. Chapter 8 summarises the results and outlines future work. Finally, Chapter A provides supplementary specifications of the input data formats, internal memory arrays, and network communication schemas.

Chapter 2

Background and Related Work

This chapter outlines the theoretical and technical foundations of the project. It starts by introducing fundamental seismological concepts and presenting the Campi Flegrei volcanic complex, which serves as the primary case study for the application. Following a review of the current state of seismic visualisation tools, the chapter concludes by explaining and justifying the software tools selected for the development of Seismic Viewer.

2.1 Seismological Fundamentals

This section introduces the basic seismological concepts needed to understand the data and analysis in Seismic Viewer, drawing from standard geophysics foundations [20].

Hypocentres and Epicentres. When an earthquake occurs, energy is released underground along a fracture. The exact point inside the Earth where the rupture begins is called the **hypocentre** (or focus). It is defined by three coordinates: latitude (ϕ), longitude (λ), and depth (z). The point directly above the hypocentre on the surface of the Earth is the **epicentre**. Traditional maps often show only the epicentre on a flat surface, losing the depth information. Seismic Viewer preserves the full three-dimensional position of each hypocentre to show the true underground structure.

Magnitude Scales. The magnitude measures the energy released by an earthquake. In local monitoring catalogues, such as those for the Campanian volcanic area, the INGV primarily uses the **duration magnitude** (M_D). Seismic Viewer can automatically detect and load different magnitude types (M_D, M_L, M_W) during data import (see Section 4.2).

Fault Geometry: Strike, Dip, and Dip Direction. Earthquakes often occur along underground fracture surfaces called faults, which can be represented geometrically as tilted planes. To describe the orientation of a fault plane in three dimensions, geologists use three fundamental angles:

The **strike** (ϕ_s) is the compass direction of the fault line. It is defined as the orientation of a horizontal line lying on the tilted plane, measured in degrees clockwise from North ($0° \leq \phi_s < 360°$).

The **dip** (δ) is the steepness or inclination of the fault plane. It is the angle between the tilted plane and a horizontal surface, measured along the line of maximum slope perpendicular to the strike. The dip angle ranges from 0° for a completely horizontal plane to 90° for a vertical fault.

The **dip direction** is the compass direction toward which the fault plane slopes downward into the subsurface. It is always perpendicular to the strike:

$$\text{Dip Direction} = (\phi_s + 90°) \bmod 360° \tag{2.1}$$

Together, strike and dip define the complete 3D position of a geological fault. In Seismic Viewer, these angles are calculated automatically from earthquake clusters by fitting planes using the RANSAC [6] and SVD [19] algorithms (described in Chapter 6).

2.2 The Campi Flegrei Volcanic Complex

The Campi Flegrei caldera, situated west of Naples in southern Italy, is an active volcanic area in Europe.

Since 2005, Campi Flegrei has experienced ground uplift (*bradyseism*) accompanied by seismic swarms [3]. The INGV Osservatorio Vesuviano monitors the area through seismic stations, GNSS/GPS receivers, and geochemical sensors. The resulting datasets, containing thousands of earthquakes, ground movement series, and tomography data, form the main validation case for Seismic Viewer.

Studying subsurface processes at Campi Flegrei, such as fluid movement and fault activation, requires combining multiple data sources within a shared spatial and temporal framework. This requirement motivated the layered design of Seismic Viewer (see Section 5.5). Recent studies [7] also showed that earthquake locations in this caldera concentrate along planar surfaces, supporting the integration of plane estimation methods (RANSAC and SVD).

2.3 Existing Seismic Visualisation Tools

A variety of software tools are used in seismology, ranging from general Geographic Information Systems (GIS) to specialized geophysical software. However, existing solutions exhibit trade-offs between domain-specific functionality, interactivity, and ease of use.

QGIS and ArcGIS. These general-purpose GIS platforms excel at 2D cartography, spatial overlay analysis, and integrating seismic events with surface infrastructure and geological maps. However, their core architecture is designed for 2D raster and vector layers rather than dense, dynamic 3D point clouds. Although 3D views can be rendered natively or via plugins like Qgis2threejs, performance drops significantly when handling tens of thousands of hypocentres. In addition, they lack built-in tools

for interactive 4D time-windowed playback and automated structural fault plane fitting.

GMT and PyGMT. The Generic Mapping Tools (GMT) and its Python interface (PyGMT) represent the standard choice for producing high-resolution static maps, cross-sections, and focal mechanism plots. However, GMT is fundamentally a batch-processing, script-driven tool that generates static image files (such as PostScript or PNG). It does not provide an interactive graphical environment where an analyst can rotate 3D scenes in real time, interactively adjust clustering parameters, or observe dynamic earthquake migrations.

ObsPy. ObsPy is the standard open-source Python framework for seismological research, offering comprehensive tools for waveform processing, event metadata management, and standard file format support (such as MiniSEED, SAC, and QuakeML). While ObsPy is widely used for data ingestion and signal processing, it is a programming library rather than an interactive desktop application. Its visualisation capabilities rely mainly on static 2D Matplotlib figures.

Seisan. Seisan is a widely used software suite for earthquake observatory workflows, including phase picking, hypocentre location, and seismic database management. While reliable for routine monitoring, its user interface and codebase are based on legacy terminal utilities and Fortran routines, presenting a steep learning curve and lacking modern, hardware-accelerated 3D/4D visual exploration capabilities.

ParaView. ParaView is an open-source scientific visualisation engine based on VTK, capable of rendering large 3D and 4D volumetric datasets, such as seismic tomography models and wave propagation simulations. However, ParaView is a general-purpose scientific visualisation tool: it requires complex data pipeline configurations and lacks domain-specific seismological features, such as direct online catalog ingestion (e.g., the INGV GOSSIP API), automated hypocentral clustering, and strike/dip fault plane extraction.

GPlates. GPlates is an interactive plate tectonics tool designed for reconstructing geological continental motions across global scales and deep geological time (millions of years). While it supports 3D spheres and time animation, it is designed for global plate kinematic models rather than high-resolution, local earthquake sequences or real-time volcano-seismic monitoring.

Table 2.1 summarises the capabilities of these established tools and highlights the functional gap addressed by Seismic Viewer.

The comparison confirms that no existing software combines interactive 3D rendering, temporal 4D animation, automated machine learning clustering, real-time parameter tuning, and structural fault plane fitting within an accessible desktop application. Seismic Viewer was designed specifically to unify these capabilities.

Table 2.1. Comparison of existing seismic visualisation tools against Seismic Viewer.

Tool	3D View	4D Anim.	Cluster	Real-time	GUI	Primary Limitations
QGIS	Limited	×	Plugin	×	✓	General GIS, not optimised for dynamic 3D point clouds
GMT / PyGMT	Static	×	×	×	×	Static script-based outputs, 3D views are rendered as fixed images, not interactive scenes
ObsPy	Limited	×	External	×	×	Library framework, no interactive desktop GUI
Seisan	Limited	×	×	×	Partial	Legacy observatory suite, steep learning curve
ParaView	✓	✓	×	Partial	✓	General-purpose VTK tool, lacks seismology workflows
GPlates	✓	✓	×	Partial	✓	Global plate tectonic focus, not local earthquake swarms
Seismic Viewer	✓	✓	✓	✓	✓	Dedicated 4D seismotectonic analysis in a standalone app

2.4 Machine Learning in Seismology

The application of machine learning to seismological data has grown substantially in recent years, establishing data-driven analysis and clustering as primary tools for interpreting complex seismic sequences [1, 11] and supported by computational libraries such as scikit-learn [14]. Two families of algorithms are particularly relevant to the spatial analysis of hypocentral distributions, and Seismic Viewer integrates both to offer complementary analytical perspectives.

Density-Based Clustering: DBSCAN. The Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm [4] identifies clusters as areas of high point density separated by areas of low density. Unlike partitioning methods, DBSCAN does not require specifying the number of clusters in advance and can identify structures of arbitrary shape, which is useful when analysing non-planar fault zones. The algorithm uses two parameters: ε (the search radius) and `MinSamples` (the minimum number of points required to form a dense group). Points outside these clusters are marked as noise, filtering out background events to isolate structural features.

Centroid-Based Clustering: K-Means. The K-Means algorithm [10] groups N points into K clusters by minimising the sum of squared distances to each cluster center. Seismic Viewer uses the **MiniBatchKMeans** variant, which processes random subsets (mini-batches) of data at each step. This reduces computation time and enables fast updates as the user adjusts parameters. K-Means is suitable when the number of target structures is known or when spherical partitions are desired.

Temporal Tracking: Cluster Label Switching. When clustering point clouds over time, unsupervised algorithms run independently on each animation frame

and may assign different labels (and colors) to the same physical structure across successive time steps. In an animation, this causes colors to change between frames. To preserve consistent colors across temporal sequences, the system uses a label matching algorithm, described in Section 6.4.

2.5 Technology Choices and Justifications

To choose the technology stack, various alternatives were systematically compared, focusing on a balance between high performance, rapid development, and future maintainability. Table 2.2 summarises the key decisions.

Table 2.2. Summary of the selected technology stack and rejected alternatives.

Component	Chosen	Rejected	Justification
Language	Python 3.10+	Java, C++	Rich ecosystem for data science, rapid prototyping
Desktop Framework	pywebview [21]	Electron [12], Qt	Lightweight memory footprint, native file dialogs, direct Python to JS bridge
HTTP Server	FastAPI [16]	Flask, Django	Native async support, WebSockets, automatic API documentation, high throughput
Rendering	Plotly.js [15]	D3.js, Three.js, CesiumJS	High-level 3D API, native WebGL acceleration, built-in interactivity
Clustering	scikit-learn [14]	Custom, R	Reliable and highly optimized implementations, includes MiniBatchKMeans and DBSCAN
Maps	Smopy [17] (OSM [13])	Mapbox, Google Maps	Free and open-source, requires no API key, supports offline caching

2.5.1 Architectural Trade-Off: pywebview vs. Electron

Selecting the desktop framework affects memory use and execution speed. While **Electron** [12] is widely used for desktop web apps, it bundles a browser and Node.js with the application. This configuration uses around 200 MB of memory in idle and requires communication between Node.js and Python.

In contrast, **pywebview** [21] uses the web component already present in the operating system (WebView2 on Windows, WebKit on macOS). This keeps memory usage low and avoids bundling an extra browser engine. In addition, pywebview provides a direct bridge (`js_api`), allowing user actions and file dialogs to run without network overhead.

2.5.2 Rendering Engine Selection: Plotly.js vs. Low-Level Graphics APIs

Rendering 3D earthquake datasets interactively requires GPU acceleration with WebGL. Low-level graphics engines like **Three.js** provide control over visual elements, but require writing 3D axes, projections, legends, and camera controls from scratch. Conversely, 2D charting tools like **D3.js** lack 3D WebGL support.

Plotly.js [15] combines WebGL acceleration with built-in plotting features and interactive controls. By using Plotly's WebGL pipeline and preserving camera orientation during updates (`uirevision`), Seismic Viewer achieves smooth rendering for large earthquake catalogs without requiring a custom graphics engine.

2.5.3 Backend Server Framework: FastAPI vs. Traditional Frameworks

The backend transfers binary data buffers and handles WebSocket messages alongside numerical calculations. Synchronous Python web frameworks like **Flask** or **Django** can block incoming requests during long calculations unless coupled with external task queues.

FastAPI [16], running on the `uvicorn` [2] server, uses an asynchronous event loop via `asyncio`. This design supports WebSocket communication and HTTP endpoints within a single process. Calculations are offloaded to background threads or process pools, keeping the user interface responsive.

Chapter 3

System Architecture

This chapter describes the system architecture of Seismic Viewer, detailing its structural decomposition, communication mechanisms, and execution model. The design balances two competing demands: the computational throughput required for data processing and the responsiveness expected of an interactive desktop application.

3.1 Architectural Overview

Seismic Viewer uses a client-server desktop architecture that combines a Python scientific processing backend with an embedded web rendering engine. To avoid the memory overhead of bundling a dedicated browser runtime (such as Electron), the user interface relies on the host operating system's built-in web engine via `pywebview` [21], Apple WebKit via Cocoa `WKWebView` on macOS, Microsoft Edge WebView2 (Chromium-based) on Windows, and WebKitGTK on Linux. The backend and frontend layers communicate through three dedicated channels: a native bridge, HTTP REST endpoints, and WebSockets.

The architecture divides tasks across three dedicated communication channels, balancing responsiveness and data transfer speed:

1. **Native Bridge**: Connects the user interface with the Python runtime for desktop actions. When the window opens, pywebview exposes methods of the Python `Api` class to JavaScript via `window.pywebview.api`. Actions such as selecting files or opening dialogs call these methods directly without network delay.

2. **HTTP REST Endpoints**: Handles data transfer between backend and frontend. Rather than embedding large datasets into HTML pages, the frontend requests catalogs asynchronously. Numerical coordinates are streamed as Float32 binary buffers via `/api/data/{id}/bin`, while catalog metadata and views are served with standard compression.

3. **WebSockets**: Manages live updates and streaming during long calculations. When running temporal animations or clustering routines, the backend streams progress notifications and frame updates directly to the client over an open connection, updating the interface without page reloads.

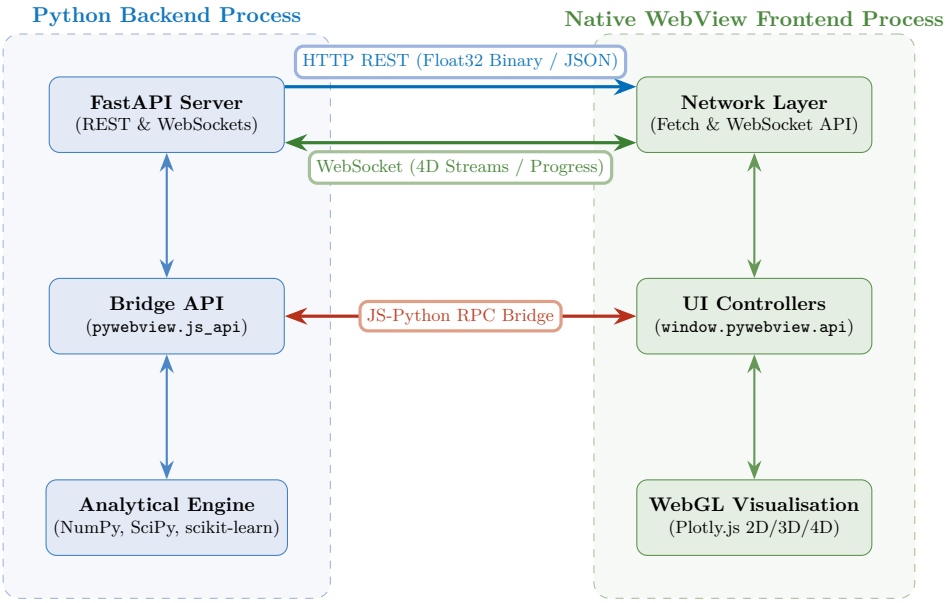


Figure 3.1. Client-server architecture of Seismic Viewer with dedicated communication channels.

Table 3.1 lists the REST endpoints and WebSocket message types supported by the system. The full JSON schemas for these network messages are detailed in Section A.2 of Chapter A.

Table 3.1. REST API endpoints and WebSocket message protocol.

Protocol / Endpoint	Direction / Path	Description / Message Types
REST (`GET`)	`/api/data/{session_id}`	Retrieve session data as GZip-compressed JSON
REST (`GET`)	`/api/data/{session_id}/bin`	Retrieve numerical data as Float32 binary buffer
REST (`GET`)	`/viz/{viz_id}`	Serve pre-rendered HTML visualisation page
WebSocket	Client → Server	`ping`, `get_cluster_update`, `get_structural_planes`, `get_animated_clustering`
WebSocket	Server → Client	`pong`, `rpc_response`, `progress` (broadcast messages)

3.2 Backend Architecture

3.2.1 Application Entry Point & Bridge Engine

Once the application is running, the `Api` module acts as the central coordinator. Whenever the user interacts with the interface, such as selecting a file (`select_csv()`), loading a 3D view (`load_view()`), or exporting results (`export_clusters_csv()`), the frontend sends a request directly to this coordinator.

These requests involve computationally intensive spatial analysis calculations on seismic events. To prevent blocking the graphical interface during execution, operations are delegated to background worker threads or process pools. A critical engineering challenge is safely transmitting live telemetry and progress notifications back to the frontend without causing race conditions or deadlocks between synchronous compute workers and the asynchronous network loop. The coordinator solves this by dispatching WebSocket broadcast coroutines onto the FastAPI event loop via `asyncio.run_coroutine_threadsafe()`. This pattern guarantees thread-safe message dispatching, ensuring the user interface receives real-time progress updates without locking the main window loop or corrupting socket state.

3.2.2 Session Store & FastAPI Middleware

The system architecture requires a dedicated memory management layer to preserve state between user interactions and to serve data efficiently to the graphical frontend. This functionality is centralized within an internal configuration module (`core/config.py`), which orchestrates two separate in-memory caches: `_data_store` and `_html_store`.

The `_data_store` structure retains the raw numerical arrays and metadata associated with each active analytical session. To guarantee that the application maintains a stable memory profile even during extensive analytical workflows, this cache is strictly limited to 30 concurrent sessions. In parallel, the `_html_store` caches up to 15 pre-rendered graphical views, allowing the frontend to load complex tridimensional scenes instantaneously. Both structures implement a First-In, First-Out (FIFO) eviction policy. To ensure safety during simultaneous operations, updates to these stores are synchronized, keeping application memory use low during long sessions.

Positioned between this caching layer and the client interface, the FastAPI server deploys specific middleware to optimize network throughput. An important optimization is the automatic application of GZip compression for any outgoing data payload larger than 500 bytes. Compressing the data prior to transmission accelerates the rendering process, particularly when transferring datasets containing tens of thousands of seismic events.

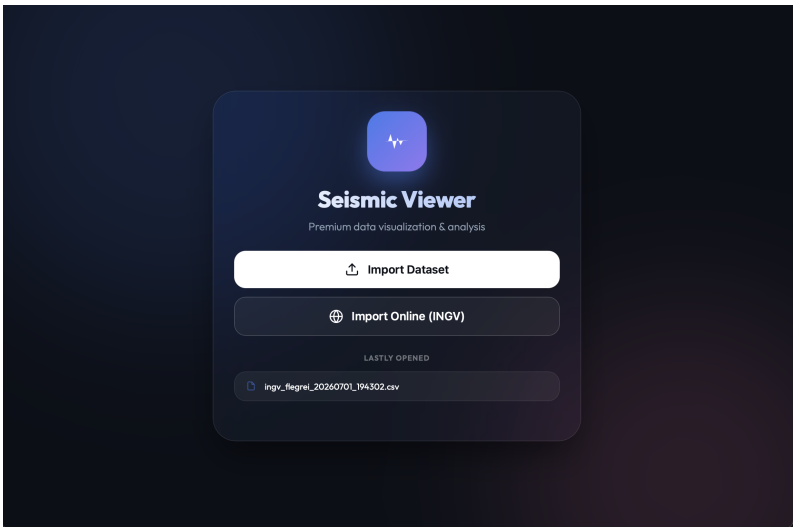
3.3 Frontend Architecture

3.3.1 Single-Page Application & UI/UX Design

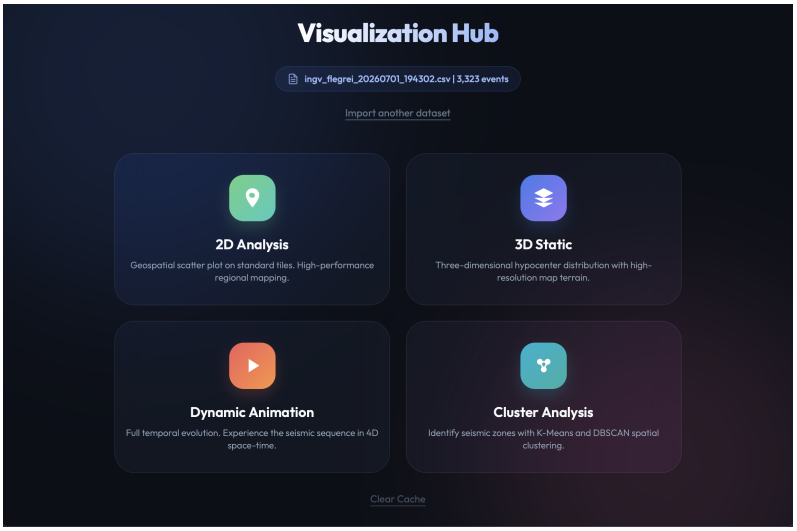
As illustrated in Figure 3.2, the primary workflow begins at the landing screen (a). Here, the system exposes multiple data ingestion pathways, allowing researchers to import local csv files, query the INGV remote web services directly, or restore a previously interrupted session through local historical records.

Once the imported dataset is validated, the software opens the visualization hub (b). This view provides menus to switch between the available display modes and clustering algorithms. In addition, it includes toggles to display extra data layers on the 3D map, such as seismic tomography, ground uplift, and monitoring stations.

The 3D graphics pipeline delegates rendering to WebGL through Plotly.js inside the native WebView component (WebKit or WebView2). This avoids the memory overhead of shipping a dedicated browser binary while providing access to hardware-accelerated 3D graphics. To preserve context during updates, the system retains the view angle and zoom level selected by the user. This prevents the camera from resetting when new data arrives, maintaining smooth interaction (Section 7.2.1).



(a) File Picker view.



(b) Visualisation Hub with mode selection.

Figure 3.2. The Seismic Viewer single-page application structure, showing initial data loading (a) and the multi-mode analytical selection hub (b).

3.4 Data Flow Pipeline and Protocols

To manage data processing and rendering, Seismic Viewer divides operations between backend services and the frontend client. On the backend, the system handles data loading, validation, and mathematical calculations. As shown in Figure 3.3, the pipeline begins by importing CSV data or querying the INGV API. This data is cleaned and loaded. Visual pre-computations prepare coordinate structures before the engine executes the selected algorithm. Finally, results are stored in the internal caches and rendered in HTML.

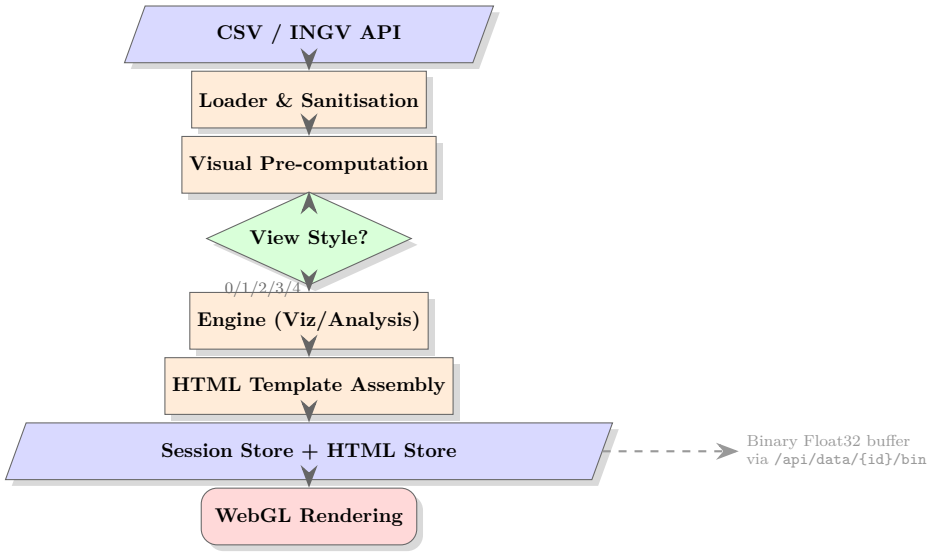


Figure 3.3. Linear data pipeline from input parsing to WebGL drawing.

Once the backend processing is complete, the frontend retrieves the data through the defined REST API and WebSocket endpoints (Table 3.1). The frontend dynamically renders the 3D scene using WebGL via the Plotly.js library. Continuous status updates, real-time recalculations, and filtering commands are exchanged over the WebSocket channel, ensuring the interface remains fully interactive without needing page reloads (Figure 3.4).

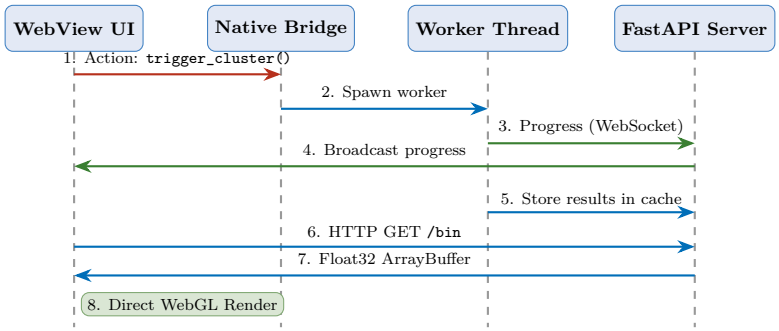


Figure 3.4. Sequence diagram of asynchronous execution and non-blocking streaming pipeline.

3.4.1 Binary Data Transfer Protocol

To eliminate JSON parsing overhead during large catalog transfers, the backend packs numerical coordinates into a contiguous Float32 buffer (`/api/data/{id}/bin`) structured as: header `[event_count, cluster_count]` followed sequentially by arrays for longitude, latitude, magnitude, depth, `dates_num`, and cluster labels (C floats), as summarized in Table 3.2. On the frontend, decoding via JavaScript's `Float32Array` directly over an `ArrayBuffer` completes in a few milliseconds even for large datasets, reducing transfer volume by approximately 80% and avoiding JSON parsing delays.

Table 3.2. Binary buffer layout (Float32).

Offset	Content	Offset	Content
0 (2 floats)	Header: `[N, C]`	$2+3N$ (N floats)	Depth array
2 (N floats)	Longitude array	$2+4N$ (N floats)	Dates_num array
$2+N$ (N floats)	Latitude array	$2+5N$ (C floats)	Cluster labels (if present)
$2+2N$ (N floats)	Magnitude array		

3.5 Threading and Concurrency Model

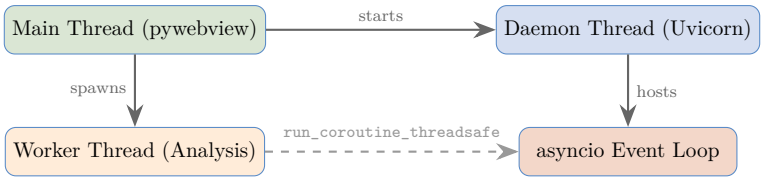


Figure 3.5. Concurrency hierarchy linking UI, server thread, and analysis worker threads.

The concurrency architecture divides responsibilities across the **Main Thread** (hosting the user interface managed by `pywebview`), the **Server Thread** (running the Uvicorn [2] event loop), and background **Worker Threads** for data analysis (Figure 3.5).

Because Python's standard `asyncio` event loop is not thread-safe, calling WebSocket functions directly from a background worker thread can cause conflicts and errors. To solve this, the backend sends notifications using `asyncio.run_coroutine_threadsafe(coroutine, loop)`. This function places the coroutine into a thread-safe queue. The Uvicorn server thread then sends the progress updates to the frontend without interrupting the data processing.

For long calculations, such as animated clustering and plane tracking across multiple time windows, the engine uses a `ProcessPoolExecutor` with up to 8 worker processes. Running tasks across separate processes avoids Python Global Interpreter Lock limitations and utilizes multiple CPU cores. This keeps the application responsive while processing large datasets.

Chapter 4

Data Processing and Geospatial Engine

This chapter details the processing pipeline that transforms raw seismic observations (whether sourced from local catalogs or remote web services) into optimized, GPU-ready structures for interactive 3D and 4D visualization.

4.1 Pipeline Architecture and Multi-Source Ingestion

Seismic Viewer uses a unified data pipeline to import, validate, and convert seismological and geophysical data for 3D and 4D visualization. As illustrated in Figure 4.1, the architecture consists of three main stages:

1. **Multi-source ingestion**: The application receives seismic catalogs from local CSV files, queries the INGV GOSSIP API for remote datasets, and loads auxiliary geophysical layers such as seismic tomography grids and GNSS station coordinates.

2. **Catalogue sanitisation**: Incoming earthquake records pass through a validation sequence that checks spatial fields, flags missing metadata, enforces coordinate and depth limits, removes future dates, and sorts events in chronological order.

3. **Unified memory and streaming**: Validated records are converted into contiguous numerical arrays in memory and packed into Float32 binary buffers for hardware-accelerated rendering.

The following sections describe each component of this pipeline in detail.

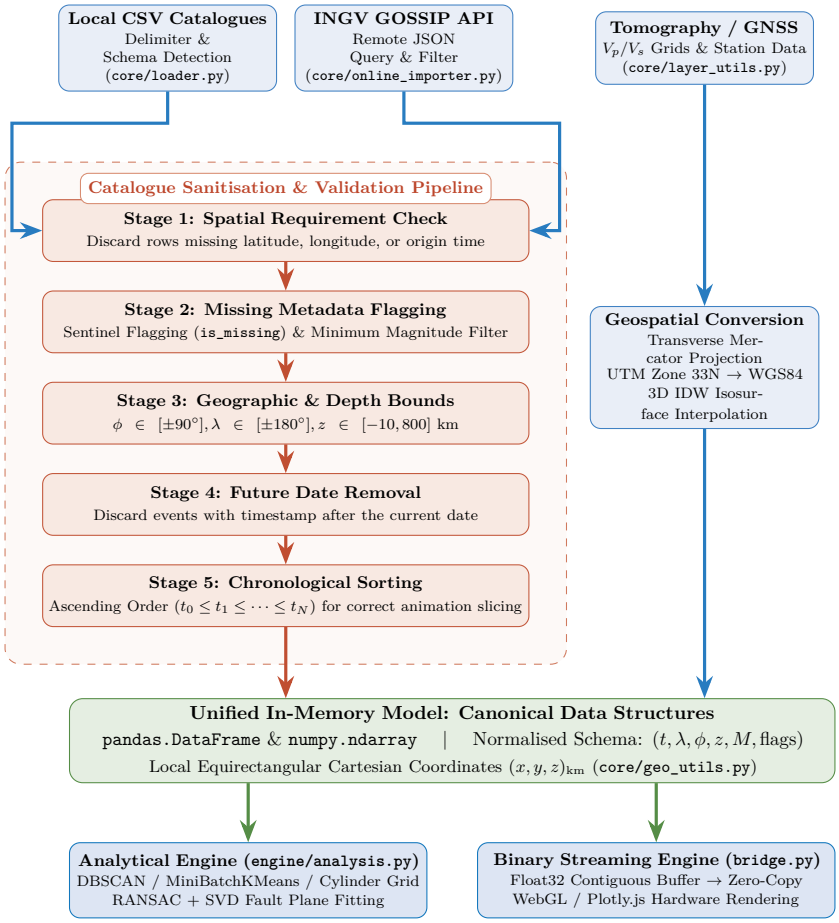


Figure 4.1. Data ingestion pipeline of Seismic Viewer, showing data sources (Local CSV, INGV GOSSIP API, and Tomography or GNSS data), the validation pipeline, and data conversion into memory arrays and binary buffers.

4.2 CSV Ingestion and Schema Detection

The `core/loader.py` module handles different formats through an ingestion pipeline that checks the header line for comma or semicolon delimiters, supporting European standards (INGV) as well as international formats (USGS, ISC). The expected catalog schema is documented in Table A.1 of Chapter A. To avoid manual formatting, `core/config.py` defines an alias dictionary:

```
CSV_SCHEMA = {
    'time':  ['time', 'date', 'datetime', 'utc', 'time_utc', '
    origintime'],
    'lat':   ['lat', 'latitude'],
    'lon':   ['lon', 'longitude'],
    'depth': ['depth', 'depth_km'],
    'mag':   ['mag', 'magnitude', 'md', 'ml', 'mw']
}
```

Listing 4.1. Flexible column schema definition in `core/config.py`.

The ingestion pipeline identifies data columns by reading their headers. It ignores text case and removes any extra spaces. When a file contains unlabelled columns, the loader assumes a standard INGV column order based on their positions. Date processing is equally robust. The engine tests multiple known formats, covering both standard ISO timestamps and common European layouts.

4.3 Data Sanitisation and Validation

After ingestion, data records pass through a validation and cleaning sequence:

1. **Spatial requirement check**: Rows missing latitude, longitude, or timestamps are discarded immediately.

2. **Missing metadata flagging**: Catalogues occasionally omit depth or magnitude. In seismic analysis, estimating missing depths (such as using the mean value) is avoided because it groups points into artificial horizontal layers, distorting clustering and plane fitting results. Rather than dropping these records (which removes spatial density) or imputing synthetic values, missing fields are flagged via boolean indicators (`depth_is_missing`, `mag_is_missing`) and assigned sentinel defaults (0.0 km depth and magnitude 0.1). Events with magnitude below the minimum limit (`MIN_MAGNITUDE`) are also removed. This threshold defaults to -10.0, which keeps all real seismic events. The user can raise it from the import dialog to filter out events below a chosen magnitude level.

3. **Geographic and depth bounds check**: The system excludes any event with mathematically invalid geographic coordinates (i.e., latitude outside the $\pm 90^\circ$ range or longitude outside the $\pm 180^\circ$ range). Additionally, it discards events with a depth falling outside the realistic geological range of -10 km to 800 km, where negative values are specifically permitted to accommodate earthquakes occurring above sea level within mountains or volcanic edifices.

4. **Future date removal**: Events whose timestamp lies after the current date are discarded, as they indicate erroneous catalog entries.

5. **Chronological sorting**: The remaining events are sorted in ascending time order to ensure correct slicing during animation and filtering operations.

4.4 Online Data Import: The INGV GOSSIP API

The `core/online_importer.py` module interfaces directly with the INGV GOSSIP web service [9], providing high-resolution volcanic monitoring catalogues for Campi Flegrei, Vesuvius, and Ischia:

The importer queries the official INGV GOSSIP web portal for each selected year (using the endpoint `https://terremoti.ov.ingv.it/gossip/<network>/<year>/events.json`). It downloads these annual JSON packets one by one to avoid network failures. As shown in Figure 4.2, users can define the network target, the time span, the magnitude limits (M_{min} to M_{max}), and the desired INGV revision quality

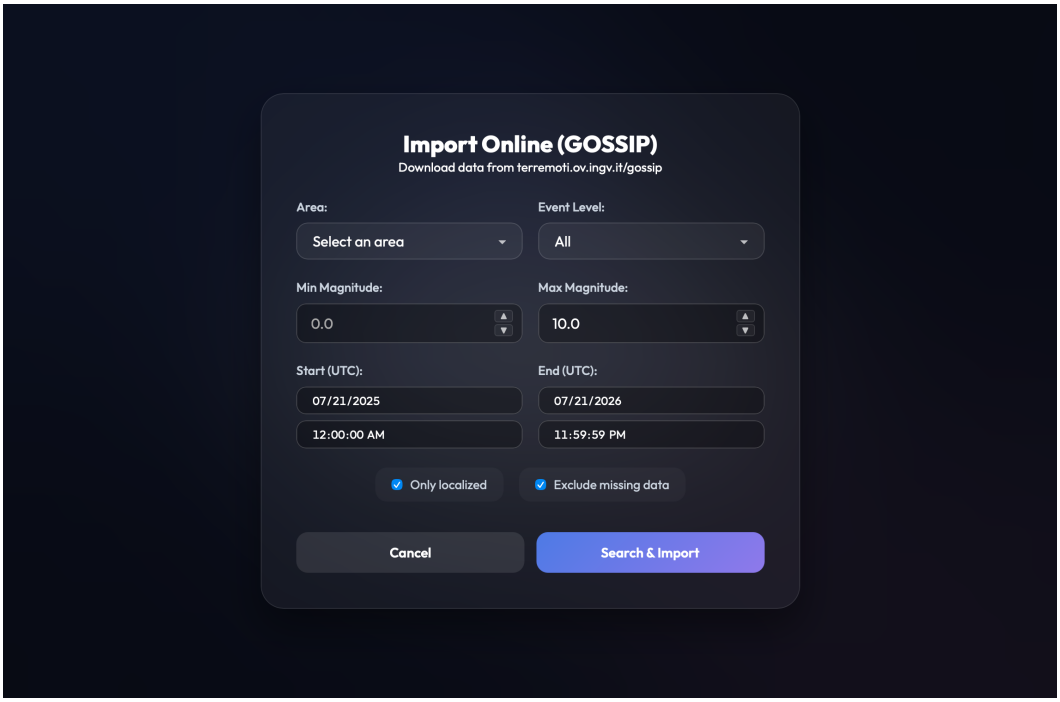


Figure 4.2. The INGV GOSSIP API import dialog for configuring network criteria, temporal bounds, and quality thresholds.

(`automatico`, `rivisto`, or `bollettino`). Users can also apply additional filters to require confirmed coordinates (`located only`) or to drop records with incomplete information (`exclude missing`). Finally, the system converts these filtered records into a standard temporary CSV format and forwards them directly to the main ingestion pipeline.

4.5 Geospatial Computations and Mesh Generation

The 3D application visualises seismic activity over a base map retrieved from OpenStreetMap [13]. To accurately draw the earthquakes over this map, the software must convert the image pixel positions into standard geographic coordinates. The system solves this problem using an **inverse Mercator projection** [22] (implemented in `core/geo_utils.py`). This mathematical operation translates pixel data into latitude and longitude, using NumPy [8] arrays to guarantee fast execution:

$$\lambda = \frac{x \cdot s}{t \cdot n} \cdot 360^\circ - 180^\circ, \quad \phi = \arctan\left(\sinh\left(\pi\left(1 - \frac{2 \cdot y \cdot s}{t \cdot n}\right)\right)\right) \cdot \frac{180^\circ}{\pi} \tag{4.1}$$

In these equations, λ and ϕ represent the resulting longitude and latitude in degrees. The values x and y correspond to the horizontal and vertical pixel coordinates on the screen. The variable s is the current scaling factor of the map, while t represents the fixed dimension of a single map tile (256 pixels). Lastly, n is the total number of tiles across the map at the current zoom level z, calculated as $n = 2^z$.

4.5.1 UTM Zone 33N to WGS84 Conversion

Some advanced datasets, such as volcanic tomography grids, are originally provided in UTM Zone 33N coordinates instead of standard latitude and longitude. To ensure these layers align correctly with the rest of the 3D map, the `core/layer_utils.py` module converts them using a Transverse Mercator inverse projection on the WGS84 ellipsoid:

$$\phi = \phi_1 - \frac{N_1 \tan \phi_1}{R_1} \left(\frac{D^2}{2} - \frac{(5 + 3T_1 + 10C_1 - 4C_1^2 - 9e'^2)D^4}{24} \right) \tag{4.2}$$

Here, ϕ is the final converted latitude. The formula relies on several variables derived from the WGS84 ellipsoid model. Specifically, ϕ_1 is the footprint latitude, N_1 is the prime vertical radius of curvature, and R_1 is the meridional radius of curvature at ϕ_1. The parameter D represents the scaled distance from the central meridian, calculated as $D = (E - 500{,}000)/(N_1 k_0)$, where E is the UTM Easting coordinate and k_0 is the central scale factor. Finally, T_1, C_1, and e'^2 are standard geometric constants related to the Earth's eccentricity.

4.5.2 Optimised Map Mesh and Caching Architecture

Terrain generation operates via a five-stage sequence: (1) bounding box squaring for uniform axis resolution, (2) deterministic cache lookup via MD5 hashing of rounded geographic bounds, (3) OpenStreetMap tile downloading via Smopy [17] on cache misses, (4) palette-indexed downsampling to 1024×1024 px (256 colours), and (5) grid transformation via Equation (4.1). Meshes are cached both in RAM (`@lru_cache(maxsize=4)`) and disk (`.pkl`), guaranteeing sub-millisecond retrieval on subsequent loads.

4.6 Pre-computation and Memory Layout Transformation

To keep the 3D view fast, the application calculates visual data in advance during data loading. The `precompute_visual_data()` function prepares popup texts and converts dates into numerical values (days from the first event). This allows quick color mapping over time without recalculating during user interaction.

The pipeline stores coordinates and depths into NumPy arrays (`np.float32`). Transmitting numbers as raw bytes instead of text JSON avoids conversion overhead and reduces data size.

To avoid large HTML files, the application loads earthquake data in two steps. First, the backend creates an HTML page with the 3D map layout and controls. Next, the browser downloads the earthquake points in the background via `/api/data/{id}/bin` as a Float32 buffer. This keeps page loading fast and responsive.

Chapter 5

Visualisation Modes

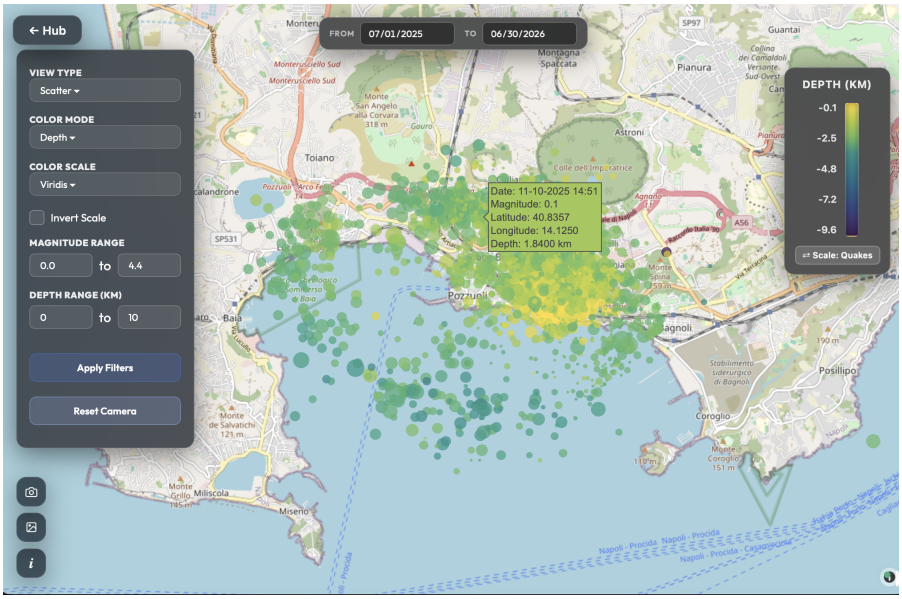
Seismic Viewer provides four primary visualisation modes, each addressing a distinct analytical need. This chapter documents their implementation, rendering strategies, and integration with auxiliary geophysical data layers.

5.1 2D Geospatial Map View

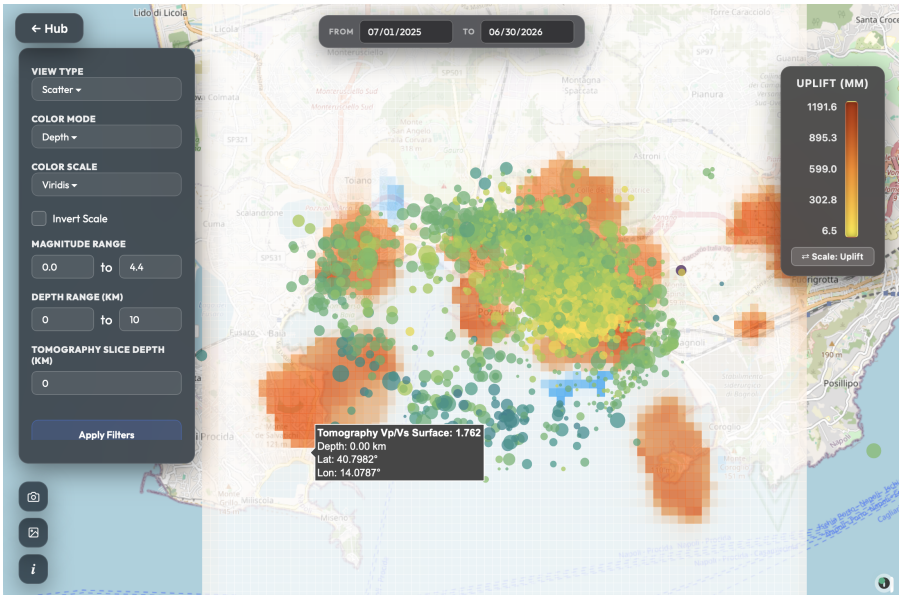
The 2D view provides a plan-view representation of earthquake epicentres overlaid on OpenStreetMap [13] tiles, implemented through Plotly's `scatter_mapbox` trace type. The map visualises seismic data using two distinct graphical layers:

- **Scatter Markers**: Each earthquake is represented by a circular marker. The visual properties of these circles can be mapped to different physical parameters depending on the analytical needs. By default, the size of the circle indicates the magnitude of the event (with a minimum size threshold to ensure extremely weak micro-earthquakes remain visible), and the color represents the hypocentral depth. However, these mappings are highly flexible, allowing users to visualize other variables, such as chronological progression. Users can interact with these markers by hovering over them to reveal detailed statistics.
- **Density Heatmap**: A spatial density layer that visually aggregates tightly clustered earthquakes. By rendering a continuous color gradient over regions of high seismicity, this optional overlay allows researchers to intuitively identify active swarm locations and broader structural trends that might be difficult to perceive when analyzing individual scatter points alone.

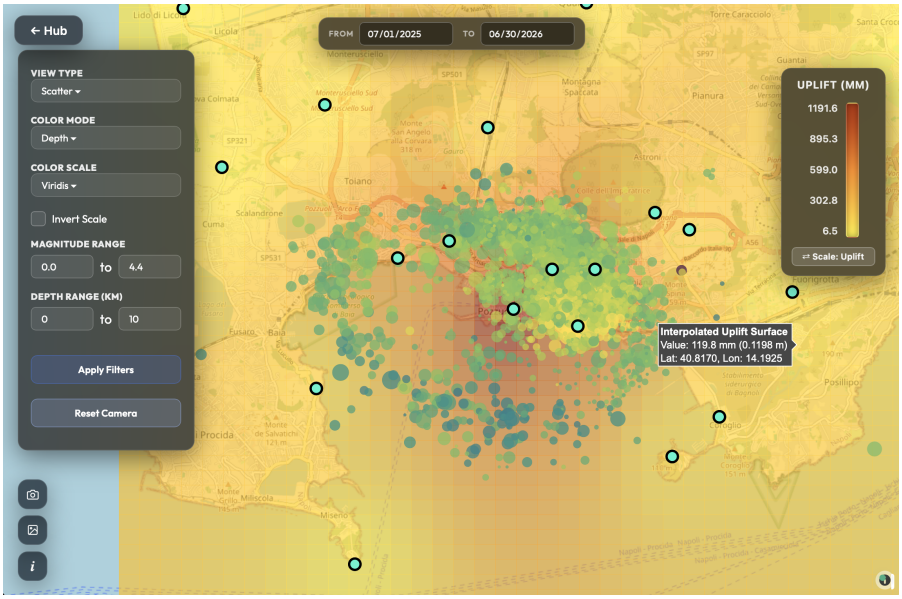
To explore the dataset, users can apply filters based on magnitude, depth, and date ranges using dedicated input fields located on the left-hand panel. All filtering operations are processed directly within the user's web browser. This approach avoids sending new requests to the main server, guaranteeing that the map updates instantly whenever a parameter is changed.



(a) The 2D base map showing earthquake locations. On the left panel, input fields allow the user to filter the data by magnitude, depth, and date.



(b) An overlay showing a tomography slice of the crust. The highlighted areas indicate high V_p/V_s velocity anomalies.



(c) An overlay displaying vertical ground movement (uplift) using a grid map, along with the locations of permanent monitoring stations across Campi Flegrei.

Figure 5.1. The 2D Geospatial Map View in the Seismic Viewer. This workspace allows users to filter earthquake data (a) and combine it with advanced geophysical overlays, such as tomographic velocity maps (b) and ground uplift measurements (c).

As shown in Figure 5.1, the 2D workspace offers three visualization options. The base view (Figure 5.1a) displays earthquake locations on an OpenStreetMap layer, while the side panel allows users to filter events by magnitude, depth, and date. In Figure 5.1b, a tomographic slice shows variations in the V_p/V_s velocity ratio, indicating underground fluid bodies. Finally, in Figure 5.1c, the map combines monitoring stations with vertical ground movement, helping researchers observe if earthquakes match areas of surface uplift.

5.2 3D Static View

The 3D static view renders hypocentres as a subsurface point cloud within a three-dimensional geographic volume beneath an OpenStreetMap terrain surface. The reference terrain is rendered as a flat, unshaded map layer for clarity, while earthquakes are represented beneath the surface as a 3D point cloud, colour-coded by variables like depth or time.

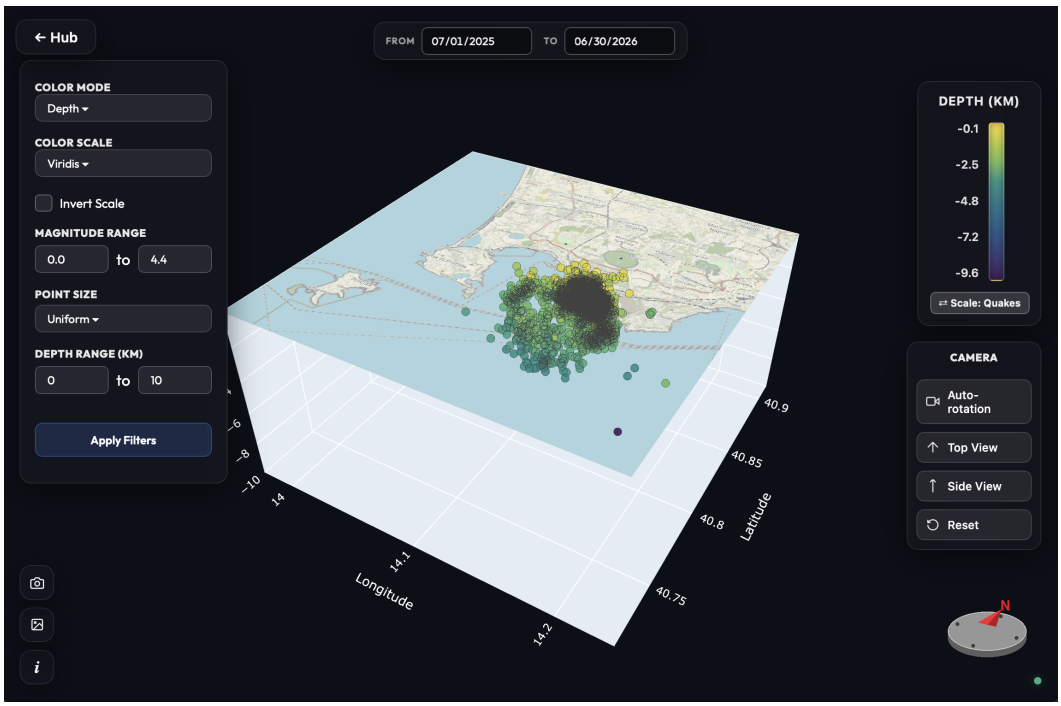
To prevent geographic distortion caused by Earth's sphericity, the system scales horizontal proportions based on latitude:

$$\text{ratio}_x = \frac{\Delta\lambda \cdot K \cdot \cos(\bar{\phi})}{\max(d_{\text{lon}}, d_{\text{lat}})}, \quad \text{ratio}_y = \frac{\Delta\phi \cdot K}{\max(d_{\text{lon}}, d_{\text{lat}})} \tag{5.1}$$

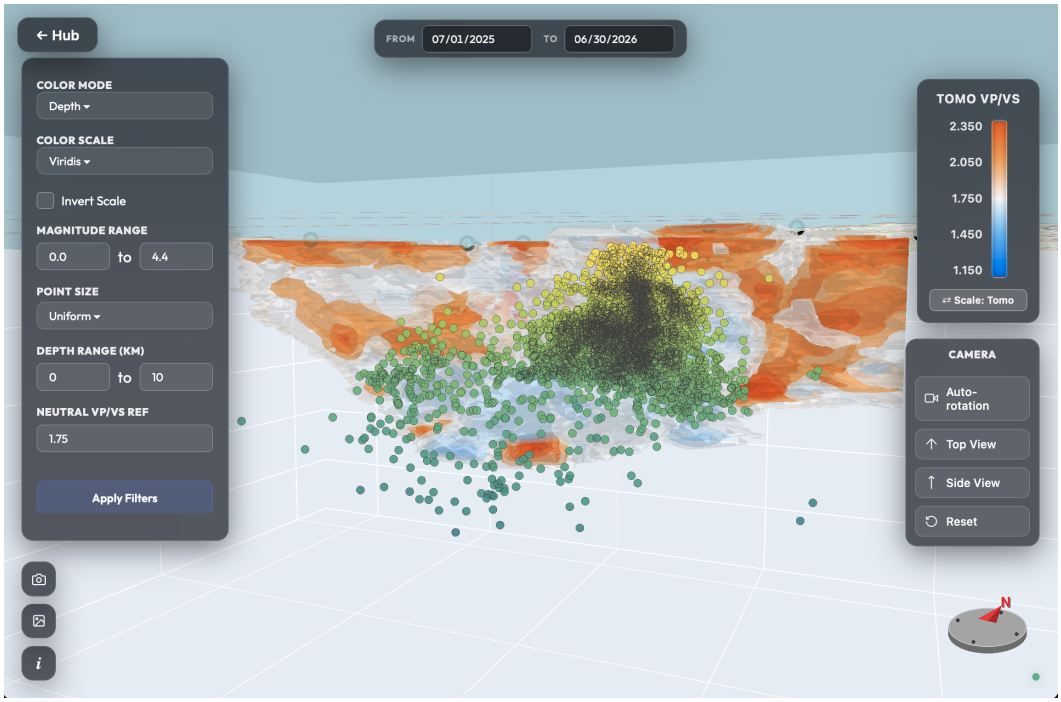
where $\Delta\lambda$ and $\Delta\phi$ are the longitude and latitude spans, $K = 111.32$ km/° is the equatorial conversion factor, and $\bar{\phi}$ is the mean latitude. This adjustment preserves accurate spatial proportions.

Rendering large point clouds simultaneously relies on GPU acceleration. Plotly.js uses WebGL to draw points via the `scatter3d` trace type. To maintain smooth interactive frame rates, the application applies three measures: (1) map background images are scaled to save memory, (2) automatic bounding box recalculations are disabled during updates, and (3) markers are drawn as point sprites facing the camera rather than complex 3D shapes.

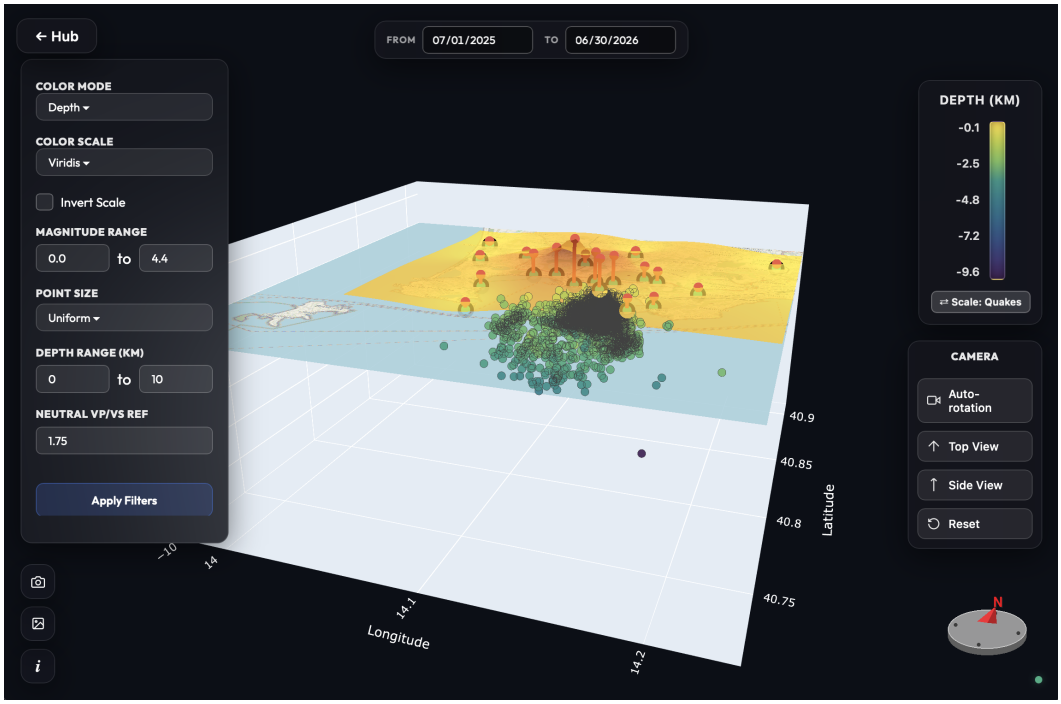
As shown in Figure 5.2, the 3D static workspace presents these data layers in three dimensions. The base view (Figure 5.2a) shows hypocentres below the map surface, using colours to represent depth intervals. In Figure 5.2b, the scene adds velocity slices from the tomography model. Finally, in Figure 5.2c, vertical columns indicate surface movement at each station, allowing direct comparisons between depth and surface observations.



(a) Point cloud below surface map.



(b) Added tomography layer.



(c) Ground uplift columns.

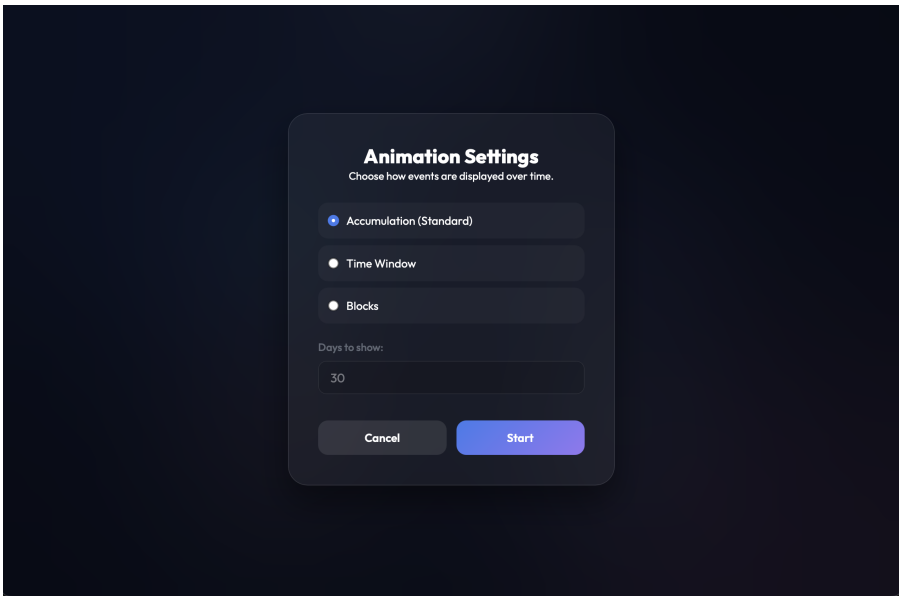
Figure 5.2. The 3D Static View workspace. The user can explore the underground earthquake distribution (a) and combine it with 3D tomography models (b) and surface deformation data (c).

5.3 3D Animated View (4D Visualisation)

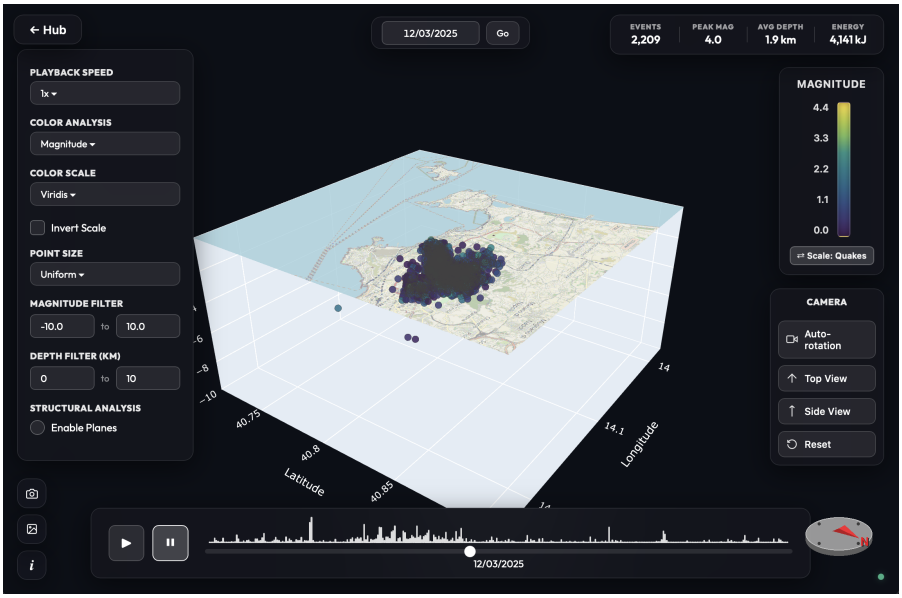
The animated view adds time as a fourth dimension to observe the spatio-temporal evolution of seismic sequences.

The 4D visualisation engine offers three temporal modes to analyse seismic evolution: **Accumulation** (showing all events up to the current frame), **Sliding window** (displaying events within a moving time frame), and **Block mode** (rendering discrete, non-overlapping time intervals). To ensure smooth playback and avoid memory overload, the system precomputes temporal indices and updates the 3D scene dynamically without reloading the entire figure.

During 4D animations, the camera angle and zoom level must remain stable across updates. By default, Plotly reinitializes the view on coordinate changes. Setting `uirevision='constant'` in the layout preserves camera position and orientation across frames, allowing continuous interaction during playback.



(a) Temporal window configuration dialog.



(b) Active 4D playback view showing the time-sliced hypocentral point cloud.



(c) Dynamic cylinder grid partitioning with automated structural plane fitting during 4D evolution.

Figure 5.3. The 4D Animated Spatio-Temporal View workflow. Users configure parameters via the dialog (a), explore time-evolving seismicity (b), and observe automated fault plane tracking (c).

Prior to starting the animation, the time parameters (such as step size and frame rate) are configured via a dedicated dialog window (Figure 5.3a). The resulting 4D view (Figure 5.3b) allows interactive exploration of the chronological sequence. In addition, the volume can be partitioned into a spatial grid of vertical cylinders (Figure 5.3c). This subdivision was implemented to better understand the spatial distribution of the data, as partitioning the volume effectively isolates and subdivides the complex fault geometry into local, simpler planar segments. As earthquakes occur inside these cylinders, the system dynamically calculates and renders these local structural fault planes in real-time, allowing users to visualize how seismic events evolve over time across the identified planar structures.

5.4 3D Cluster Analysis View

The cluster analysis mode integrates spatial machine learning with structural geological modeling.

5.4.1 Cluster Visualisation and Interactive Tuning

To provide a comprehensive geometrical understanding of seismogenic structures, Seismic Viewer renders every identified cluster through four complementary representations within the 3D WebGL scene. These representations were chosen to address different analytical needs: boundary delineation, volumetric extent, structural orientation, and individual event membership. Together, they allow researchers to examine a cluster from multiple geometric perspectives simultaneously.

1. **Wireframe Convex Hulls (`Scatter3d`)**: Outer boundary envelopes highlighting the spatial footprint and volumetric extent of each distinct cluster.

2. **Triangulated Solid Meshes** (`Mesh3d`, with `alphahull=0`): Smooth, semi-transparent volumetric convex hulls enclosing the clusters.

3. **Structural Fault Planes (`Mesh3d`)**: Planar surfaces calculated via RANSAC robust estimation coupled with Singular Value Decomposition (SVD) refinement. Each plane displays orientation vectors alongside precise geological strike and dip angles upon mouse hover.

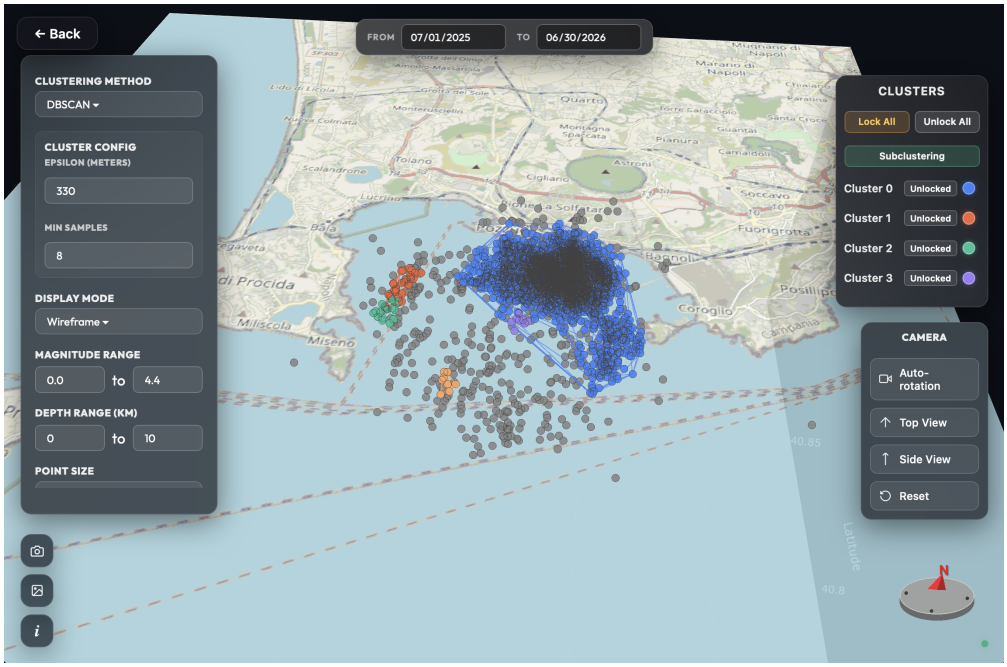
4. **Clustered Event Spheres**: Discrete event spheres colored by cluster label using categorical palettes, with unclustered background noise explicitly assigned neutral grey.

When users interactively adjust spatial clustering hyperparameters, such as neighborhood radius ε, minimum samples threshold `min_samples`, or cluster count K, via the side control panel, the frontend dispatches a `get_cluster_update` WebSocket message. The Python backend asynchronously recomputes cluster labels and structural planes without blocking the event loop, returning the newly generated Plotly trace definitions. The frontend then replaces the existing traces in the WebGL scene, thereby updating the visualization while preserving the user's current camera perspective.

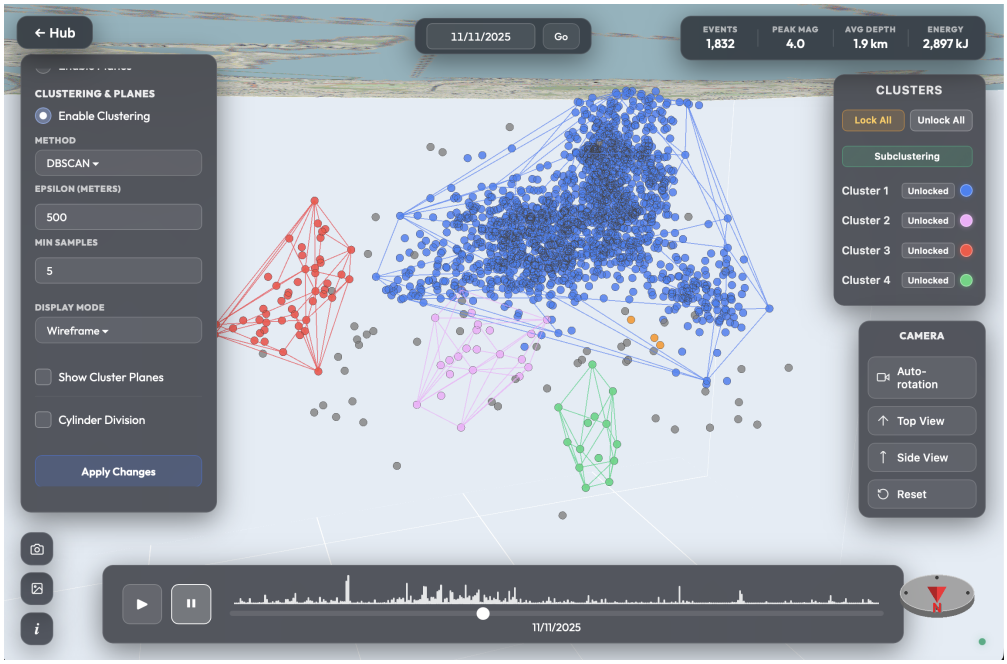
5.4.2 Sub-Clustering

To analyze hierarchical structures, researchers can perform secondary sub-clustering on specific data subsets. By utilizing the **locked clusters** feature, users can preserve previously identified and validated groups. The algorithm will then selectively re-analyze and sub-divide all the remaining unlocked clusters simultaneously. The refined results can subsequently be applied back to merge cleanly into the parent view.

Figure 5.4 provides a comprehensive overview of the clustering tools. Figure 5.4a shows the standard 3D clustering interface. Figure 5.4b displays how clustering evolves over time. To ensure visual consistency, a tracking algorithm guarantees that each cluster maintains the same color across all animation frames. Finally, Figure 5.4c and Figure 5.4d combine the animated clusters with dynamically calculated fault planes and spatial cylinders. This combination allows researchers to easily observe how geological structures evolve and rotate over time.



(a) Static 3D workspace with hulls and planes.

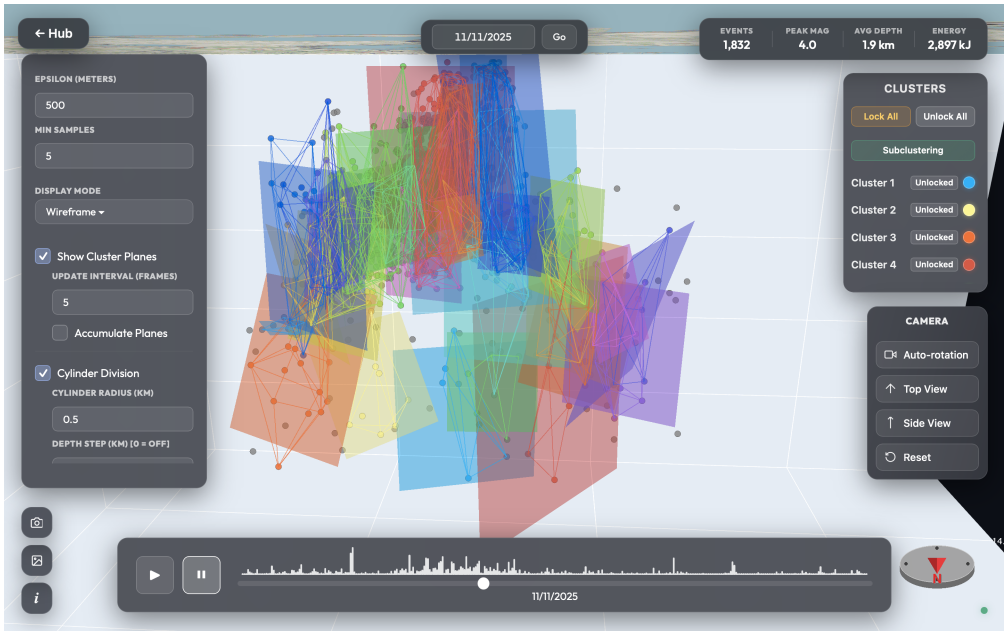


(b) Animated clustering with Global Label Matching.

Figure 5.4. Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 1).



(c) Animated clustering with RANSAC+SVD planes.



(d) Unified 4D cylinder partitioning and plane fitting.

Figure 5.4. Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 2).

5.5 Additional Data Layers & View Caching

5.5.1 Auxiliary Layers: Tomography, Uplift, and Stations

Seismic Viewer integrates three auxiliary data layers, processed through the `core/layer_utils.py` module:

- **Seismic Tomography (V_p/V_s Ratio)**: The raw tomographic data, originally provided in UTM Zone 33N coordinates, is converted to geographic coordinates and mapped onto a regular 3D grid. Because measurements only exist at specific surveyed points, the software must estimate the values for the surrounding empty space. It achieves this using *Inverse Distance Weighting*, a spatial interpolation technique. The value of any unknown point is calculated as a weighted average of its $k = 16$ nearest known neighbors. The closer a neighbor is, the stronger its influence on the final calculation, scaling inversely with the square of the distance ($p = 2$):

$$V(\mathbf{x}) = \frac{\sum_{i=1}^{k} w_i \cdot V_i}{\sum_{i=1}^{k} w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)^2} \tag{5.2}$$

  However, Inverse Distance Weighting can produce unrealistic mathematical artifacts in empty regions located very far from actual measurement stations. To solve this, the system applies a *distance-decay* function, which acts as a spatial fade-out effect. As the distance from the nearest station increases, this function smoothly blends the estimated value back into the standard natural background value of the Earth's crust (1.75):

$$V_{\text{final}}(\mathbf{x}) = \alpha(\mathbf{x}) \cdot V_{\text{IDW}}(\mathbf{x}) + (1 - \alpha(\mathbf{x})) \cdot 1.75 \tag{5.3}$$

  where $\alpha(\mathbf{x}) = \exp(-(d_{\min}/\sigma)^2)$ with $\sigma = 0.15$ acts as the blending weight, decreasing smoothly from 1 (near known data) to 0 (far away). The value $\sigma = 0.15$ degrees ($\approx$17 km) was chosen to match the typical distance between survey stations in the Campi Flegrei dataset, so that the fade-out occurs at the same scale as the station spacing.

- **GNSS Ground Deformation (Uplift)**: To visualize ground movement, the system reads historical displacement records and matches them with the known locations of the monitoring stations. This provides the exact vertical movement at any given time. Because data is only available at these specific points, the software estimates the ground movement in the empty areas between them using the same distance-based averaging (IDW) described above. The final result is displayed on the map either as a colored 2D grid or as 3D columns rising from the surface.

- **Monitoring Stations**: Permanent station positions are plotted as labelled markers, aiding spatial evaluation of catalogue completeness thresholds.

These three auxiliary layers serve complementary roles in the interpretation workflow. By overlaying tomographic anomalies and ground deformation patterns onto the same 3D scene as the earthquake hypocentres, researchers can visually

assess whether seismic swarms correlate with known velocity anomalies or with areas of maximum surface uplift, providing a more integrated understanding of the underlying volcanic processes.

5.5.2 View Caching and Navigation

To ensure faster navigation between complex workspaces, the backend bridge maintains an in-memory view cache (`_html_store`, max 15 entries) keyed by a composite tuple of file path, view style, animation/windowing parameters, clustering configuration, and locked cluster states. A navigation history stack (`_history_stack`) allows instantaneous back and forward navigation in the clustering view while preserving exact parameter context.

Chapter 6

Clustering and Structural Analysis Algorithms

This chapter formalises the algorithms used in Seismic Viewer's analytical engine, presenting their mathematical formulations and optimization strategies.

6.1 Coordinate Projection and Spatial Clustering Methods

To compute distances for spatial clustering, spherical coordinates (λ, ϕ) are converted into a local Cartesian coordinate system in kilometres $(x, y, z)_{\text{km}}$ centered at the dataset mean (λ_0, ϕ_0). This projection [22] is defined as:

$$x = (\lambda - \lambda_0) \cdot K \cos(\phi_0), \quad y = (\phi - \phi_0) \cdot K, \quad z = z_{\text{depth}} \tag{6.1}$$

where $K = 111.32$ km/° represents the average length of one degree of latitude, and z_{depth} represents the depth of each individual earthquake (measured as positive downwards).

6.1.1 DBSCAN, MiniBatchKMeans, and Cylinder Partitioning

Seismic Viewer implements three complementary clustering paradigms, building on machine learning techniques for seismic catalog analysis [1] and provided by the scikit-learn library [14] or developed as custom extensions:

1. **DBSCAN**: The density-based spatial clustering algorithm [4] (`DBSCAN`) isolates high-density seismogenic structures according to neighbourhood radius $\varepsilon_{\text{km}} = \varepsilon_{\text{m}}/1000$ and `MinSamples`. Points lacking sufficient density reachability are marked as noise (label -1).

2. **MiniBatchKMeans**: For centroid partitioning, the MiniBatchKMeans variant [10] optimises K clusters across random data subsets ($b = 1024$), reducing iteration complexity from $O(NK)$ to $O(bK)$. To prevent outlier inclusion, a post-clustering distance filter reclassifies points exceeding $d_{\max}$ from their centroid as noise:

```
if method == 'kmeans' and max_dist > 0:
    max_d = max_dist / 1000.0
    for i in range(n_clusters):
        idx = (labels == i)
        dists = np.sqrt(np.sum((pts_km[idx] - centroids[i])
    **2, axis=1))
        display_labels[np.where(idx)[0][dists > max_d]] = -1

```

Listing 6.1. Post-clustering distance filter for KMeans.

3. **Cylinder Partitioning**: To ensure stable cluster assignments during 4D temporal animations without color swapping, the study volume is partitioned into a regular grid of vertical cylinders. Events are assigned to the nearest cylinder centroid via KDTree query. Cylinders accumulating more than 5 events constitute stable, depth-resolved clusters across frames.

Table 6.1. Comparison of spatial clustering approaches in Seismic Viewer.

Property	**DBSCAN**	**MiniBatchKMeans**	**Cylinder Partition**
Cluster count / shape	Automatic / Arbitrary	User-defined (K) / Spherical	Grid-dependent / Cylindrical
Noise handling	Native (label -1)	Distance threshold ($d_{\max}$)	Density threshold (≥ 5 events)
Primary use case	Unknown structures, swarms	Fixed centroid tracking	4D animation, fault plane tracking

6.2 Automatic Parameter Tuning

Parameter tuning is executed during *Subcluster Analysis* to estimate parameters for unlocked clusters based on their spatial distribution. The method is based on the Kneedle algorithm [18], which detects the point of maximum curvature in a sorted distance or inertia curve.

DBSCAN Auto-ε. Density-based clustering depends on two parameters: the search radius ε and the minimum number of points k = MinSamples. Because these parameters depend on each other, the analytical engine determines both automatically. First, the minimum sample threshold is computed from the dataset size N as $k = \max(3, \lfloor \ln N \rfloor)$, allowing the density requirement to adapt to the data volume. Next, using a nearest neighbor search, the algorithm calculates the distance $d_i^{(k)}$ from each earthquake to its k-th closest neighbor. Sorting these distances in ascending order yields the k-distance curve, denoted as points $\mathbf{p}_i = (i, d_i^{(k)})$ for $i = 0, \ldots, N-1$.

In spatial clustering theory, points belonging to seismic clusters exhibit lower distances, whereas boundary events and noise cause the curve to rise. The search

radius ε^* corresponds to the knee point of this curve, which marks the transition between cluster density and noise. To identify this point, the system constructs a line between the first point $\mathbf{p}_0 = (0, d_0^{(k)})$ and the last point $\mathbf{p}_{N-1} = (N-1, d_{N-1}^{(k)})$, defining the direction vector $\mathbf{v} = \mathbf{p}_{N-1} - \mathbf{p}_0$. The algorithm calculates the distance from each curve point $\mathbf{p}_i$ to the line and selects the index i^* that maximizes this distance:

$$i^* = \arg\max_{0 \le i < N} \left\| (\mathbf{p}_i - \mathbf{p}_0) - \left(\frac{(\mathbf{p}_i - \mathbf{p}_0) \cdot \mathbf{v}}{\|\mathbf{v}\|^2} \right) \mathbf{v} \right\| \tag{6.2}$$

The search radius is assigned as $\varepsilon^* = d_{i^*}^{(k)}$, providing a parameter that separates earthquake clusters from background noise.

MiniBatchKMeans Auto-K. For centroid-based clustering, finding the number of clusters K requires balancing group compactness against model complexity. The analytical engine evaluates values from $K = 2$ to $K = \min(10, N)$, computing the clustering inertia $J(K) = \sum_{j=1}^{K} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$ for each configuration. Plotting $J(K)$ against K produces a decreasing curve. The cluster count K^* is identified by applying the projection method described in Equation (6.2) to the inertia curve, locating the elbow point where adding clusters yields diminishing improvements in compactness.

6.3 Structural Plane Fitting and Geometric Synthesis

To extract planar fault structures from earthquake distributions, Seismic Viewer uses a two-stage process combining Random Sample Consensus (RANSAC) [6] and Singular Value Decomposition (SVD) [19]. RANSAC isolates points belonging to a plane while removing noise. SVD then computes the best-fitting plane across the identified inliers and evaluates the planarity of the structure.

6.3.1 RANSAC Robust Plane Estimation

Seismic earthquakes rarely form perfect planes and contain background events. Fitting a plane directly using least squares is skewed by outliers. RANSAC addresses this through an iterative consensus search. In each step, the algorithm samples three points $\{P_1, P_2, P_3\}$ and calculates a candidate normal vector:

$$\vec{n}_{\text{cand}} = \frac{(P_2 - P_1) \times (P_3 - P_1)}{\|(P_2 - P_1) \times (P_3 - P_1)\|} \tag{6.3}$$

The algorithm counts how many points in the cluster lie within a distance threshold of this candidate plane. Points meeting this condition are classified as inliers. This process is repeated across iterations, and the candidate model with the largest inlier count and lowest residual error is selected as the consensus set.

6.3.2 SVD Refinement and Strike/Dip Mechanics

Although RANSAC removes outliers and isolates the inlier set, the candidate plane itself is defined by three sample points. To obtain the plane that accounts for all identified inliers, the engine applies Singular Value Decomposition (SVD).

The software computes the centroid $(\bar{x}, \bar{y}, \bar{z})$ of all n inliers and centers the coordinates at the origin, constructing the data matrix $A \in \mathbb{R}^{n\times 3}$:

$$A = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} & z_1 - \bar{z} \\ \vdots & \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} & z_n - \bar{z} \end{bmatrix} = U\Sigma V^T \tag{6.4}$$

The decomposition factors A into orthogonal vectors and singular values $\Sigma = \mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)$, where $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. The singular vector corresponding to the smallest singular value σ_3 defines the normal vector $\vec{n}$ of the fitted plane.

Beyond determining the plane orientation, the singular values provide direct insight into the geometric shape of the cluster. Following standard 3D point-cloud dimensionality metrics used in structural geology [5], the analytical engine evaluates a normalized Planarity index P:

$$P = \frac{\sigma_2 - \sigma_3}{\sigma_1} \tag{6.5}$$

A high planarity score ($P \to 1$) indicates a planar fault segment, whereas a low score ($P \to 0$) denotes an isotropic or linear dispersion of seismic hypocenters.

To translate the normal vector $\vec{n} = [n_x, n_y, n_z]$ into standard geological angles, the vector is oriented downwards ($n_z \geq 0$). Trigonometric formulas then compute the orientation of the fault:

$$\text{Dip}: \quad \delta = \arccos(n_z) \cdot \frac{180^\circ}{\pi} \tag{6.6}$$

$$\text{Dip Direction}: \quad \theta_{\mathrm{DD}} = \mathrm{atan2}(n_x, n_y) \bmod 360^\circ \tag{6.7}$$

$$\text{Strike}: \quad \phi_s = (\theta_{\mathrm{DD}} - 90^\circ) \bmod 360^\circ \tag{6.8}$$

The Dip measures the tilt angle relative to the surface, the Dip Direction indicates the direction of the downward slope, and the Strike defines the orientation of the horizontal fault line. For 3D visualization, planes are rendered as triangulated grids along orthogonal basis vectors $\vec{u}, \vec{v}$:

$$\vec{u} = \frac{\vec{n} \times \hat{z}}{\|\vec{n} \times \hat{z}\|}, \quad \vec{v} = \frac{\vec{n} \times \vec{u}}{\|\vec{n} \times \vec{u}\|} \tag{6.9}$$

6.4 Temporal Stability, Parallel Execution, and Complexity

Computing structural models across temporal sequences requires visual consistency and computational efficiency. This section describes the temporal tracking mechanism, the parallel execution model, and the computational complexity of the analytical engine.

6.4.1 Temporal Stability via Global Label Matching

When running unsupervised clustering algorithms frame by frame during 4D animations, each time step evaluates an independent subset of seismic events. Because unsupervised algorithms assign cluster labels without historical context, the label of a fault structure can change between consecutive frames. In an animation, this causes color changes (known as color flickering), which prevents tracking how faults evolve over time.

To maintain color continuity across frames, the engine uses a label matching algorithm. The system maintains an array recording the most recent label assigned to each earthquake. For every cluster detected in a new frame, the algorithm inspects the previous labels of its events and applies majority voting:

$$L_{\text{assigned}} = \arg\max_{k} \sum_{j \in S_c} \mathbb{I}(\text{global_map}[j] = k) \tag{6.10}$$

where S_c is a sample of indices from the cluster (bounded to 200 events), and $\mathbb{I}$ is the indicator function. If a cluster contains only new events without previous history, the algorithm assigns a new label. The assigned label is applied to color the earthquakes in the active frame and saved in the global array so that subsequent frames reuse the same cluster identifier.

6.4.2 Parallel Execution Architecture

Calculating RANSAC inliers and SVD planes across animation frames involves separate mathematical steps for each time window. To prevent these calculations from slowing down the user interface, the backend distributes frame tasks across CPU cores using a process pool. The coordinate array is stored once in worker memory at startup, allowing worker processes to read coordinates directly without making copies.

6.4.3 Asymptotic Complexity Analysis

Table 6.2 summarizes the time and space complexity of each algorithm in the pipeline. Spatial neighbor searches in DBSCAN use a KD-Tree index to reduce query complexity to $O(N \log N)$, while MiniBatchKMeans scales with mini-batch size rather than full dataset dimensions. Plane fitting scales with the number of inliers per cluster, and label matching remains bounded by sampling.

Table 6.2. Asymptotic time and space complexity of the analytical methods.

Algorithm / Method	Time Complexity	Space Complexity	Parameters & Notation
DBSCAN (KD-Tree)	$O(N \log N)$	$O(N)$	N events, ε neighbourhood radius
MiniBatchKMeans	$O(b \cdot K \cdot I)$	$O(b \cdot K)$	$b = 1024$ batch, K centroids, I iterations
Cylinder Partitioning	$O(N \log M)$	$O(M + N)$	M grid cylinders (60×50)
Kneedle Auto-Tuning	$O(N \log k + K_{\max} \cdot b)$	$O(N)$	k-NN ($k = \max(3, \lfloor \ln N \rfloor)$), $K \in [2, 10]$
RANSAC + SVD Fitting	$O(I_{\text{ransac}} \cdot N_{\text{cluster}} + d^3)$	$O(N_{\text{cluster}})$	$I_{\text{ransac}} = 100$, dimension $d = 3$
Global Label Matching	$O(K \cdot \min(N_c, 200))$	$O(N)$	Majority voting with bounded sampling

Chapter 7

Development Process and Challenges

This chapter documents the software engineering process behind Seismic Viewer, detailing the development timeline, the iterative implementation methodology, major technical challenges, key architectural decisions, and quantitative performance validation.

7.1 Development Timeline & Iterative Methodology

The project evolved through continuous iterations. As depicted in Figure 7.1, development progressed through six distinct phases: (1) 3D visualization prototyping, (2) pywebview bridge integration and clustering, (3) UI/UX redesign and INGV API integration, (4) binary streaming and cylinder grid implementation, (5) integration of tomography and GNSS station layers, and (6) modular package refactoring and testing.

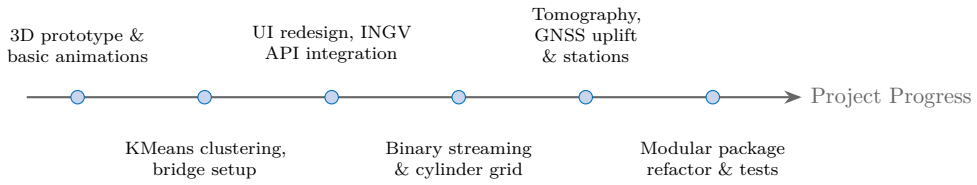


Figure 7.1. Development milestones highlighting the iterative implementation methodology.

The software was built using an iterative workflow, with features prototyped and refined through continuous feedback. As the application expanded, the codebase was refactored twice: first by splitting a single script into a modular package, and later by introducing FastAPI and WebSockets for data streaming.

7.2 Key Technical Challenges and Solutions

Developing a desktop seismological analysis application combining a Python backend with a browser rendering engine presented software engineering challenges. This

section analyzes the main technical bottlenecks encountered during development, the architectural solutions designed to solve them, and their impact on system stability and responsiveness.

7.2.1 Rendering Performance and UX Optimisation

The graphical interface must handle large point clouds with interactive frame rates while maintaining a smooth user experience.

Camera State Preservation across Dynamic Re-clustering. In early prototypes, interactive operations such as re-clustering, parameter adjustments, or depth filtering triggered a complete view reset. Because the Plotly.js engine re-evaluates the 3D scene when receiving updated traces, the virtual camera automatically snapped back to default coordinates. This behavior made it difficult for researchers who had aligned the view with specific fault planes. The issue was resolved by adding state tracking using the `uirevision` attribute within the figure layout (Listing 7.1). By maintaining `uirevision='constant'` across updates, Plotly preserves user orientation, elevation, zoom level, and center of rotation.

```
fig.update_layout(
    uirevision='constant',
    scene=dict(
        uirevision='constant',
        camera=dict(eye=dict(x=1.25, y=1.25, z=1.25), center=dict(x=0, y=0, z=0))
    )
)
```

Listing 7.1. Camera state persistence via `uirevision`.

JSON Deserialization Bottlenecks and Binary Streaming. Standard web communication relies on JSON text formatting for data exchange. However, transferring dense earthquake catalogs with many events exposed performance limits. In Python, converting large numerical arrays into JSON text created CPU overhead. In addition, the browser engine paused while parsing large JSON strings, dropping the frame rate. To remove this bottleneck, the architecture was redesigned around a raw Float32 binary streaming protocol (Section 3.4.1). Coordinates, cluster labels, and metadata are packed into a continuous binary buffer on the backend and sent directly to the client. On the frontend, the payload is read into a native `Float32Array` typed array. This shift reduced data transfer size by about 80% and significantly lowered loading times, maintaining smooth interaction.

WebGL Resource Management. Switching often between views can use a lot of graphics memory. To handle GPU resources cleanly, each view transition loads a fresh page, allowing the browser to release earlier graphics buffers. In addition, earthquakes in 3D views are drawn as flat point markers facing the camera rather than solid 3D shapes. This keeps memory usage low and maintains smooth frame rates during interaction.

7.2.2 Backend Architecture, Concurrency, and Algorithmic Resilience

Managing mathematical computations, data streaming, and external web APIs within a unified desktop application required solving concurrency and stability challenges.

Native GUI Event Loop and Asynchronous Backend Coexistence. A central engineering challenge in `pywebview`-based systems is the coexistence of the native window loop and the asynchronous backend server. Native GUI toolkits demand control over the main operating system thread to process window events and user input. Simultaneously, the application backend relies on FastAPI running an asynchronous event loop for WebSockets and REST endpoints. Running both loops on the same thread causes deadlocks. To resolve this, the startup process was decoupled: the FastAPI server runs inside a background daemon thread, while `pywebview.start()` controls the main thread. Communication between threads is coordinated through thread-safe dispatchers, ensuring that backend calculations do not block the user interface.

Chromatic Stability in 4D Spatiotemporal Clustering. When unsupervised clustering algorithms (such as DBSCAN or MiniBatchKMeans) run independently on consecutive time windows during 4D animations, they assign cluster labels arbitrarily without memory of previous frames. Consequently, a single fault structure could receive different labels across frames. In an animation, this caused rapid color flickering, making it difficult to track fault propagation over time. This issue was solved by designing the Global Label Matching algorithm (Section 6.4). By maintaining a global label registry and applying majority voting across active cluster members (Equation (6.10)), the engine maps local labels to consistent global identifiers, ensuring color persistence across the entire seismic sequence.

Parallel Computation and Multiprocess Scaling. Computing RANSAC consensus inlier sets and SVD planar fits across hundreds of animation frames requires high computational effort. While Python multithreading is limited by an internal execution lock for numerical tasks, spawning standard multi-process pools introduces data copying overhead when passing coordinate arrays across processes. To reduce data duplication, the plane fitting engine uses a `ProcessPoolExecutor` with an initialization function. The coordinate array is sent once when the worker pool starts and remains stored in worker memory, preventing repeated data copies. The system then passes only index ranges to worker tasks. This approach avoids the lock and reduces communication cost, allowing linear scaling across CPU cores during animation processing.

Resilience against Remote INGV API Failures. The integrated INGV GOSSIP web service provides a REST API to query historical earthquake catalogs. However, querying multi-year seismic sequences can encounter timeouts and rate limits. In naive implementations, a single network drop would crash the entire import process. To ensure system resilience, the online importer module

(`core/online_importer.py`) divides historical queries into separate yearly requests protected by error handlers. If a specific year encounters a timeout or invalid response, the importer logs a warning, records the missing interval, and continues fetching subsequent years. The user receives a summary of imported records alongside any skipped intervals, preventing data loss without halting execution.

Session Memory Bounding and FIFO Store Eviction. During long analytical sessions, researchers frequently generate alternative 3D views, parameter variations, and sub-cluster models. Storing all generated figures and datasets in memory caused steady memory growth, which could slow down the system. To enforce a bounded memory footprint, a First-In, First-Out (FIFO) eviction policy was implemented. The engine limits `_data_store` to a maximum of 30 active session datasets and `_html_store` to 15 pre-rendered views. When capacity is reached, the oldest session objects are removed and reclaimed by memory management. The system also includes a `strip_data()` function to remove coordinate arrays from stored HTML views, reducing memory use when needed. As a result, total application memory usage remains below 500 MB even under continuous usage.

7.3 Code Quality and Architecture

The software is organized into a modular structure. The backend is written in Python across 16 modules, while the frontend is built with JavaScript, HTML, and CSS. Data flows from the entry point (`app.py`), through the bridge (`core/bridge.py`) and the analytical engine package (`core/engine/`), to the utility modules (`loader.py`, `geo_utils.py`). The engine package contains three primary modules: `analysis.py` (clustering and plane fitting), `orchestrator.py` (temporal frame orchestration), and `visualization.py` (rendering figure traces across all display modes).

A key aspect of the architectural design is the strict isolation of caching strategies. The system uses multiple caching layers operating at different levels: persistent disk caching with `pickle` for map tiles and terrain meshes, function-level caching with `lru_cache` for frequent computations, and session-scoped stores (`_data_store` and `_html_store`) for active analytical workspaces. Keeping these caches separate prevents data conflicts.

This modular organisation was not the original design. As discussed in Section 7.1, the codebase underwent two major refactoring cycles during the internship. The first split a single monolithic script into separate modules organised by responsibility, while the second introduced the FastAPI and WebSocket infrastructure to replace the original synchronous communication model. These refactoring efforts, though time-consuming, were essential for maintaining code quality as the application grew in complexity.

7.4 Validation and Quantitative Performance Assessment

The system was validated using INGV Campi Flegrei seismic datasets [9] alongside synthetic benchmark catalogs. Testing confirmed that Seismic Viewer maintains

fluid interaction and responsive frame rates even when rendering dense point clouds and multi-layer geophysical models.

All experimental benchmarks and execution latency measurements were conducted on a desktop workstation featuring an Intel Core i9-13900K processor (24 cores, 32 threads, boost clock up to 5.8 GHz), 32 GB of RAM, and an NVIDIA GeForce RTX 4070 GPU running 64-bit Windows 11.

7.4.1 Experimental Benchmarks: Serialization and Streaming

To evaluate the Float32 binary protocol against JSON, benchmarks were run across multiple dataset scales. As reported in Table 7.1, the Float32 format reduces payload size by about 80% and significantly lowers decoding time, allowing fast data transfer even on large catalogs.

Table 7.1. Serialization benchmark: JSON vs. Float32 Binary buffer.

Events (N)	JSON Size	Float32 Size	Compression	JSON Parse	Float32 Parse
1,000	95.5 KB	19.5 KB	79.5%	0.8 ms	< 0.01 ms
10,000	954.3 KB	195.3 KB	79.5%	8.1 ms	< 0.01 ms
30,000	2.80 MB	585.9 KB	79.5%	24.1 ms	< 0.01 ms
50,000	4.66 MB	976.6 KB	79.5%	40.9 ms	< 0.01 ms

7.4.2 Clustering Latency and Algorithmic Throughput

Execution times for the three clustering methods were measured across catalog sizes. For cylinder partitioning, the cylinder radius was chosen to create a grid of about 60×50 cells over the catalog area. As shown in Table 7.2, MiniBatchKMeans and Cylinder Partitioning remain fast across all tested sizes, while DBSCAN takes longer as the number of events grows.

Table 7.2. Clustering execution latency benchmark across dataset scales.

Events (N)	DBSCAN ($\varepsilon = 300$m)	MiniBatchKMeans ($K = 6$)	Cylinder Grid (60×50)
1,000	80.4 ms	68.3 ms	1.0 ms
10,000	94.9 ms	165.9 ms	5.8 ms
30,000	976.3 ms	83.0 ms	15.7 ms
50,000	2,244.0 ms	208.4 ms	22.8 ms

7.4.3 Synthetic Validation of Structural Plane Extraction

Because the true position and orientation of underground faults cannot be directly observed, the plane fitting algorithms (RANSAC and SVD) were validated using synthetic datasets where the exact geometric structures were known. These datasets were generated by mathematically defining 3D planes of points at specific strike

and dip angles, and subsequently introducing random Gaussian noise and scattered outliers to simulate realistic seismic measurement errors.

As reported in Table 7.3, the two-stage RANSAC and SVD pipeline estimates plane orientations accurately, maintaining low angular errors for both dip and strike even under significant measurement noise and outlier contamination.

Table 7.3. Synthetic plane estimation accuracy under Gaussian positional noise.

Noise σ (m)	**Outliers (%)**	**Dip MAE (°)**	**Strike MAE (°)**	**Planarity (P)**
10 m	10%	0.03°	0.04°	0.94
25 m	15%	0.06°	0.11°	0.93
50 m	20%	0.15°	0.22°	0.90
100 m	30%	0.44°	0.59°	0.84

In parallel, the global label matching algorithm (Section 6.4) successfully eliminated temporal colour flickering across 4D animated cluster sequences while maintaining fast computation times, confirming its suitability for interactive exploration workflows.

These angular errors are small relative to the uncertainty inherent in real earthquake catalogs. Standard hypocentral location methods typically produce positional errors in the range of 50–200 m [20], which correspond to angular uncertainties of several degrees when fitting planes to shallow fault zones. The errors reported in Table 7.3 ($< 0.6°$ for noise up to 100 m and 30% outliers) therefore fall within the precision that geophysical applications require, confirming that the RANSAC and SVD pipeline is suitable for fault characterisation at the scale of local seismic swarms.

Chapter 8

Conclusions and Future Work

8.1 Summary of Contributions

The analysis in Section 2.3 highlighted a gap in seismological software: no accessible tool combined 3D rendering, time animations, clustering, and structural analysis. Seismic Viewer was developed to address this need across three key areas: software architecture, data visualization, and automated analysis.

Software Architecture. A key contribution is the dual-engine design that separates computation from presentation (Chapter 3). Coupling a `pywebview` [21] window with a local `FastAPI` [16] server provides desktop integration without high memory overhead, allowing the Python backend to calculate independently without freezing the interface. Additionally, the Float32 binary streaming protocol packs numerical arrays into continuous buffers, reducing payload size by 80% compared to JSON and eliminating parsing delays during data transfer.

Data Visualization. Seismic Viewer provides four display modes: 2D maps, 3D static views, 4D animated sequences, and cluster analysis workspaces. The 4D animation mode tracks how earthquake sequences evolve across time using three temporal strategies (accumulation, sliding window, and block mode), revealing migration patterns hidden in static views. Furthermore, integrating auxiliary geophysical layers (seismic tomography, ground uplift, and stations) enables direct correlation between hypocentres and subsurface structures.

Automated Structural Analysis. The automated analytical pipeline integrates three spatial clustering algorithms: DBSCAN [4], MiniBatchKMeans [10], and cylinder grid partitioning, which prevents color changes across animation frames. Building on clustering, the system fits fault planes using RANSAC [6] and SVD [19] to calculate dip, dip direction, and strike, allowing researchers to characterize fault structures interactively.

8.2 Future Research and Architectural Extensions

While Seismic Viewer achieves interactive responsiveness for current regional catalogues, several areas present opportunities for future development.

As dataset sizes continue to grow beyond the regional scale, the current Plotly.js [15] rendering pipeline may reach its practical limits. Migrating the frontend to specialised geospatial rendering engines, such as Deck.gl or a custom WebGL pipeline built on Three.js, could enable progressive level-of-detail rendering techniques. This migration would not only increase the maximum number of renderable points but also open the possibility of deploying the application as a fully browser-based web service, eliminating the need for local installation.

A second important direction concerns real-time monitoring integration. The current architecture analyses static or periodically updated catalogues, but integrating continuous data streaming protocols such as SeedLink or FDSN web services could transform the system into a live surveillance dashboard suitable for monitoring rooms. This would require extending the WebSocket communication layer to handle continuous data ingestion alongside the existing analytical pipeline.

Finally, future iterations could expand the analytical capabilities by integrating focal mechanism visualisations to represent fault kinematics directly in the 3D scene, as well as Coulomb static stress transfer maps to analyse how individual earthquakes redistribute tectonic stress across neighbouring faults.

8.3 Concluding Remarks

Seismic Viewer demonstrates combining web technologies with Python libraries for geophysical analysis. The architecture separating the Python backend from the WebGL frontend balanced computation speed with development speed. Python libraries (NumPy [8], scikit-learn [14]) provide numerical routines, while the web frontend delivers 3D graphics.

The iterative workflow aligned software features with research requirements. Feedback from researchers at INGV Osservatorio Vesuviano and Roma Tre University guided features such as sub-clustering and cylinder partitioning.

This work supports the transition from static plots toward interactive exploration in seismology. By providing 4D visualization and clustering in a desktop application, Seismic Viewer offers a foundation for studying fault structures and monitoring volcanic unrest at Campi Flegrei [3, 7].

Appendix A

Supplementary Technical Details

This appendix provides a formal specification of the data structures and the core algorithmic logic utilized within Seismic Viewer. Standardizing the input and internal formats ensures that the system can easily parse external catalogs and efficiently render them, while the computational pipeline ensures robust data analysis. By documenting these structures and algorithms, future extensions to the software can maintain compatibility with the existing core architecture.

A.1 Seismic Data Structures

The design of Seismic Viewer places a strong emphasis on maintaining a clear separation between the external data representation and the internal state used for high-performance computation. This section outlines both formats in detail, illustrating how raw textual data is efficiently translated into numerical matrices.

A.1.1 Input Catalog Format

Seismological data must be structured in a CSV following the specifications detailed in table A.1. The schema requires specific temporal and spatial fields to ensure correct localization within the 3D WebGL environment. To accommodate the vast amount of data often recorded during seismic swarms, the format is kept as lightweight as possible. External catalogs, such as those provided by the INGV, must be pre-processed or mapped to this standard structure before ingestion. The mandatory fields include unique identifiers, precise origin timestamps, and geographical coordinates expressed in decimal degrees.

A.1.2 Internal Application State

Once parsed by the Python backend, the catalog is transformed into an internal state object optimized for rendering and clustering algorithms. The data is held in memory as multi-dimensional `numpy` arrays to support the matrix operations required by DBSCAN and RANSAC. Transitioning from a row-based CSV structure to a column-based array format significantly improves memory locality and cache performance during heavy numerical computations. This design choice helps achieve interactive frame rates in the application's user interface.

Table A.1. Expected schema for input seismic catalogs.

Field Name	Data Type	Description
`id`	`string`	Unique identifier for the seismic event
`time`	`ISO 8601 string`	Origin time of the event (e.g., 2024-01-01T12:00:00)
`latitude`	`float`	Event epicenter latitude in WGS84 decimal degrees
`longitude`	`float`	Event epicenter longitude in WGS84 decimal degrees
`depth`	`float`	Hypocentral depth in kilometers
`mag`	`float`	Magnitude of the event

Table A.2. Internal data representation in the processing pipeline.

Variable	Type	Purpose
`longitude`, `latitude`	`numpy.ndarray (float32)`	Arrays of WGS84 coordinates in decimal degrees (N events)
`depth`	`numpy.ndarray (float32)`	Array of hypocentral depths in kilometers (N events)
`magnitude`	`numpy.ndarray`	An N-length array used for scaling the visual markers
`dates_num`	`numpy.ndarray (float32)`	Days elapsed from the first event, used for temporal filtering and animation
`labels`	`numpy.ndarray`	Cluster assignments (-1 for noise, $0 \dots K$ for structural groups)
`pts_km`	`numpy.ndarray`	Array of Cartesian coordinates in kilometers ($N \times 3$), generated for clustering

A.2 Network Protocol and Message Schemas

This section provides the JSON schemas used for WebSocket communication and REST API endpoints.

A.2.1 WebSocket Message Specification

Messages sent by the client over the `/ws` WebSocket channel follow this structure:

```
{
    "type": "object",
    "required": ["type"],
    "properties": {
        "type": {
            "type": "string",
            "enum": ["ping", "get_cluster_update", "get_structural_planes", "get_animated_clustering"]
        },
        "id": {"type": "string"},
        "params": {
            "type": "object",
            "properties": {
                "method": {"type": "string", "enum": ["dbscan", "kmeans", "cylinders"]},
                "params": {
                    "type": "object",
                    "properties": {
                        "dbscan_eps": {"type": "number", "minimum": 10.0, "maximum": 5000.0},
                        "dbscan_min_samples": {"type": "integer", "minimum": 3},
                        "kmeans_clusters": {"type": "integer", "minimum": 2, "maximum": 20},
                        "locked_clusters": {"type": "array", "items": {"type": "integer"}}
                    }
                }
            }
        }
    }
}
```

Listing A.1. JSON Schema for Client-to-Server WebSocket Requests.

Progress updates and task notifications sent by the server follow this structure:

```
{
    "type": "progress",
    "task": "ransac_plane_fitting",
    "percentage": 85.0,
    "status": "Fitting cylinder planes across 120 temporal frames ...",
    "elapsed_sec": 1.42
}
```

Listing A.2. JSON Schema for Server Progress and Telemetry Broadcasts.

A.2.2 Core Default Configuration Parameters

Table A.3 summarizes the key default settings and operational thresholds adopted across the analytical engine and configuration modules:

Table A.3. Default parameters and operational thresholds adopted across the analytical engine and configuration modules.

Configuration Key	Default Value	Description / Purpose
`MAX_DATA_SESSIONS`	30	Maximum concurrent in-memory dataset sessions (FIFO eviction)
`MAX_HTML_SESSIONS`	15	Maximum cached pre-rendered HTML visualization views
`DEFAULT_DBSCAN_EPS_M`	500.0	Search radius ε in meters
`DEFAULT_DBSCAN_MIN_PTS`	5	Minimum points to define a cluster core
`DEFAULT_KMEANS_CLUSTERS`	3	Initial cluster count for MiniBatchKMeans
`CYLINDER_GRID_DIMS`	$(60, 50)$	Longitudinal and latitudinal cell resolution for 4D tracking
`RANSAC_MAX_ITER`	100	Candidate triplet sampling iterations for plane fitting
`RANSAC_THRESHOLD`	Adaptive	$\max(0.02, r_{\text{km}} \times 0.1)$, proportional to cylinder radius

Bibliography

[1] Karianne J. Bergen, Paul A. Johnson, Maarten V. de Hoop, and Gregory C. Beroza. Machine learning for data-driven discovery in solid-Earth geoscience. *Science*, 363(6433):eaau0323, 2019. doi: 10.1126/science.aau0323. URL https://doi.org/10.1126/science.aau0323.

[2] Tom Christie. Uvicorn: An ASGI Web Server for Python. https://www.uvicorn.dev.

[3] L. De Siena, G. Chiodini, G. Vilardo, E. Del Pezzo, M. Castellano, S. Colombelli, N. Tisato, and G. Ventura. Source and dynamics of a volcanic caldera unrest: Campi Flegrei, 1983–84. *Scientific Reports*, 7(1):8099, 2017. doi: 10.1038/s41598-017-08192-7. URL https://doi.org/10.1038/s41598-017-08192-7.

[4] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231. AAAI Press, 1996. URL https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf.

[5] Oscar Fernández. Obtaining a best fitting plane through 3D georeferenced data. *Journal of Structural Geology*, 27(5):855–858, 2005. doi: 10.1016/j.jsg.2004.12.004. URL https://doi.org/10.1016/j.jsg.2004.12.004.

[6] Martin A. Fischler and Robert C. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*, 24(6):381–395, 1981. doi: 10.1145/358669.358692. URL https://doi.org/10.1145/358669.358692.

[7] G. Giordano, G. Alfonsi, F. Salvini, F. Bianco, C. Chiarabba, M. Di Vito, and A. Tramelli. Birth and growth of a volcanotectonic fault during the current volcanic unrest at Campi Flegrei caldera (Italy). *Communications Earth & Environment*, 6(1):1–11, 2025. doi: 10.1038/s43247-025-02803-2. URL https://doi.org/10.1038/s43247-025-02803-2.

[8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Remi Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Krevelen, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren

Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2. URL https://doi.org/10.1038/s41586-020-2649-2.

[9] Istituto Nazionale di Geofisica e Vulcanologia. Osservatorio Vesuviano – Portale GOSSIP. https://terremoti.ov.ingv.it/gossip/.

[10] Stuart P. Lloyd. Least Squares Quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489. URL https://doi.org/10.1109/TIT.1982.1056489.

[11] S. Mostafa Mousavi and Gregory C. Beroza. Deep-learning seismology. *Science*, 377(6607):eabm4470, 2022. doi: 10.1126/science.abm4470. URL https://doi.org/10.1126/science.abm4470.

[12] OpenJS Foundation. Electron: Build Cross-Platform Desktop Apps with JavaScript, HTML, and CSS. https://www.electronjs.org.

[13] OpenStreetMap contributors. OpenStreetMap. https://www.openstreetmap.org.

[14] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[15] Plotly Technologies Inc. Plotly.js: Open Source Graphing Library for JavaScript. https://plotly.com/javascript/.

[16] Sebastián Ramírez. FastAPI: Modern, Fast Web Framework for Building APIs with Python. https://fastapi.tiangolo.com.

[17] Cyrille Rossant. Smopy: OpenStreetMap Image Tiles in Python. https://github.com/rossant/smopy.

[18] Ville Satopaa, Jeannie Albrecht, David Lo, and Matthew A. Ward. Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. In *31st International Conference on Distributed Computing Systems Workshops*, pages 166–171. IEEE, 2011. doi: 10.1109/ICDCSW.2011.20. URL https://doi.org/10.1109/ICDCSW.2011.20.

[19] Craig M. Shakarji. Least-Squares Fitting Algorithms of the NIST Algorithm Testing System. *Journal of Research of the National Institute of Standards and Technology*, 103(6):633–641, 1998. doi: 10.6028/jres.103.043. URL https://doi.org/10.6028/jres.103.043.

[20] Peter M. Shearer. *Introduction to Seismology*. Cambridge University Press, 3rd edition, 2019. doi: 10.1017/9781316877111. URL https://doi.org/10.1017/9781316877111.

[21] Roman Sirokov. pywebview: Build GUI for Your Python Program with JavaScript, HTML, and CSS. https://pywebview.flowrl.com.

[22] John P. Snyder. *Map Projections: A Working Manual*. U.S. Geological Survey Professional Paper 1395. U.S. Government Printing Office, 1987. doi: 10.3133/pp1395. URL https://doi.org/10.3133/pp1395.